

Linguaggi e Traduttori

Simone Petta

A.A. 2024/2025

Contents

1	Introduzione	3
1.1	Python	4
1.1.1	Dispatch table	5
1.1.2	Memorizzazione	5
1.2	Strutture dati	6
1.2.1	Alberi	7
1.2.2	Grafi	9
1.2.3	backtracking	11
2	Linguaggi	13
2.1	Closed form	14
2.1.1	Self embedding	14
2.2	Alberi di parsing	14
2.2.1	Pulizia di una grammatica	14
2.2.2	Dimensione degli alberi di parsing	16
2.2.3	Derivazioni	16
2.2.4	Dalla derivazione all'albero di parsing	17
2.2.5	Possibili soluzioni	18
3	Parsing	21
3.1	NPDA	22
3.2	CYK parser	24
3.2.1	Albero di parsing	27
3.2.2	Riduzione in forma normale di Chomsky	28
3.2.3	Eliminazione epsilon-regole	29
3.2.4	Eliminazione delle unit rules	31
3.2.5	Eliminazione i non solitari	32
3.2.6	Costruzione albero parsing left-most	33
3.3	Parsing Top-down, caso generale	34
3.3.1	Funzione stato prossimo	36
3.4	Parsing Ricorsivo Discendente	39
3.4.1	Eliminare Ricorsione a Sinistra	39
3.4.2	Prefix-Free	40
3.4.3	Parsing Ricorsivo Discendente	40
3.4.4	Generazione automatica del parser	42
3.5	Parsing Bottom-Up	46

3.5.1	Shift e Reduce	47
3.6	Parsing top-down direzionale deterministico	52
3.6.1	Cosa accade con le epsilon regole	55
3.7	Full LL(1)	57
3.7.1	Conflitti	62
3.7.2	Riparare i conflitti	63
3.8	Parser ricorsivo discendente predittivo	64
3.8.1	Tokenizzazione	66
3.8.2	Interprete, valutazione delle espressioni	68
3.9	Parsing deterministico bottom-up LR(0)	69
3.9.1	Automa di KNOT	69
3.9.2	Tabelle GOTO e ACTIONS	74
4	ANTLR	76
4.1	Uso diretto di ANTLR	77
4.2	Uso mediato da Liblet	78
4.2.1	Grammatiche note	78
4.2.2	Ancora su listener e visitor	79
4.3	Dall'albero di parsing all'AST	80
4.3.1	Dispatch table	80
4.4	Trasformare	80
4.4.1	Trasformare HTML in matrici	81
4.5	Traspilazione	81
4.6	Symbol table	82
4.6.1	Scope	82
4.6.2	Implementazione symbol table	82
4.7	Interpretazione	82
4.7.1	Funzioni	83
4.8	Tipi	84
4.8.1	Typechecking	84
4.9	Interprete Iterativo	85
4.9.1	Code threading	85
4.9.2	Interprete	86
5	LLVM	87
5.1	IR di LLVM	88
6	Progetto	89

Chapter 1

Introduzione

Inizio 8.45 fine 10.15

Quello che vedremo noi sono gli interpreti e in particolare riusciremo a fare un eseguibile usando un LLVM (che compila in una sorta di linguaggio macchina IR). Esiste una libreria (che ha fatto lui) che si chiama LibLet che permette di visualizzare l'esecuzione degli algoritmi che vedremo. Tutta questa cosa sarà scritta in Python senza focalizzarci sull'orientazione ad oggetti per vedere che queste cose non si possono fare solo ad oggetti. Vedremo anche un ripassone di algoritmi per vedere alcuni aspetti cruciali nel corso.

1.1 Python

Ci sono alcune strutture dati super comode che si possono usare con una sintassi comoda che sono le liste (strutture non omogenee), gli insiemi e i dizionari. Una cosa comoda di questi oggetti è che sono iterabili, in particolare è possibile costruire questi oggetti attraverso un meccanismo di comprehension in cui racchiudiamo tra i simboli sintattici la costruzione degli oggetti attraverso l'iterazione su un altro oggetto:

```
1 s = [x * x for x in range(10)]
```

Se ci metto una clausola dopo questa viene valutata:

```
1 # set comprehension: i numeri pari tra gli interi in [0, 9]
2 s = {x * x for x in range(9) if x % 2 == 0}
```

Si può imporre l'iterazione usando iter, notiamo che è comodo passare una sentinella per cui quando è finita l'iterazione ci ritorna la sentinella (tipicamente None):

```
1 #iterazione tramite iter/next
2
3 it = iter('alcune parole divise da spazi'.split())
4 while True:
5     w = next(it, None)
6     if w is None: break
7     print(w)
```

Le funzioni sono cittadini del primo ordine, possiamo assegnarle a variabili e passarle ad altre funzioni. Le useremo per implementare in modo economico i visitor, algoritmi ricorsivi che navigano le strutture dati in modo ricorsivo per fare delle cose. In più vediamo le dispatch table (un modo comodo per fare l'object oriented) e la memorizzazione tramite i decorator.

```
1 def quadra(x):
2     return x * x
3
4 def applica(fun, lst):
5     return [fun(x) for x in lst]
6
7 applica(quadra, [1, 2, 3])
```

Noi useremo molto le liste di liste, perchè ci rappresentano gli alberi e su queste possiamo definire visite.

```
1 lol = [1, [2, 3], [4, [5, 6]]]
2
```

```

3 #applichiamo una funzione scalare f a tutti gli elementi
4
5 def visit(f, lol):
6     for elem in lol:
7         if instance(elem, list):
8             visit(f, elem)
9         else:
10            f(elem)

```

1.1.1 Dispatch table

Cominciamo a vedere un piccolo esempio di parsing di un'espressione. La prima parte del parsing è suddividere la struttura lineare del testo in chunk concettuali che non è un lavoro banalissimo:

```

1  expr = "3 + 12 * 4 + 1 * 2"
2  tokens = iter(expr.split())

```

dopo di che dobbiamo definire la semantica delle operazioni, in qualche modo dobbiamo riassociare quello che osserviamo nel flusso dei token con le nostre interpretazioni. Una dispatch table è quella che associa delle informazioni a delle funzioni:

```

1 #semantica delle operazioni, tramite la dispatch table
2 def somma(x, y):
3     return x + y
4 def prodotto(x, y):
5     return x*y
6
7 DT = {
8     '+': somma,
9     '*': prodotto
10 }

```

A questo punto se voglio valutare l'espressione usiamo in modo iterativo la dispatch table, occhio che non stiamo rispettando le regole aritmetiche, ma associamo sempre a sinistra:

```

1 result = int(next(tokens))
2
3 while True:
4     t = next(tokens, None)
5     if t is None: break
6     of = DT[t]
7     result = of(result, int(next(tokens)))
8
9 result

```

1.1.2 Memorizzazione

Spesso e volentieri ci capiterà di esplorare algoritmi ricorsivi per cui per risolvere un problema con un'istanza grande risolveremo il problema su sue istanze più piccole, può accadere che nel processo di spezzamento andiamo a risolvere un sottoproblema che è già

stato risolto da qualcun altro, quindi se non adopero accorgimenti particolari ricalcolo gli stessi risultati, l'esempio tipico è il calcolo di Fibonacci.

L'idea è salvare in una cache i risultati parziali di una funzione,

```
1 def rendi_verbosa(f):
2     def f_verbosa(x):
3         result = f(x)
4         print(f'f({x}) = {result}')
5         return result
6     return f_verbosa
7
8 def quadrato(x):
9     x*x
10
11     quadrato_verboso = rendi_verbosa(quadrato)
12
13     q = quadrato_verboso(3)
14
15     #tenere da parte i risultati di una funzione
16     cache = {}
17
18     def memoize(f):
19         def f_memoized(x):
20             if x not in cache: cache[x] = f(x)
21             return cache[x]
22         return f_memoized
```

Esiste uno zucchero sintattico con cui possiamo annotare la funzione per memorizzare i risultati parziali @memoize

```
1 @memoize
2 def cubo(x):
3     return x ** 3
4
5     cache = {}
6
7     cubo(1), cubo(4), cubo(6)
8
9     cache
10
11 @memoize
12 def fib(n):
13     if n == 0 or n == 1: return 1
14     return fib(n - 1) + fib(n - 2)
```

1.2 Strutture dati

Le strutture dati che vedremo sono:

- alberi
- grafi
- pile/code

Gli alberi li useremo per rappresentare il testo che parseremo utilizzando una gerarchia del testo, e per visitare questa gerarchia utilizzeremo le visite. Un altro algoritmo che useremo è il backtracking che è molto utile per comprendere gli algoritmi di parsing, è una tecnica ricorsiva che tenta di risolvere i problemi con una sorta di forza bruta ma senza infilarci nelle chiamate ricorsive anche inutili.

1.2.1 Alberi

Gli alberi sono la struttura dati più comune del corso perchè c'è una forte ricorrenza tra linguaggi/parsing e alberi, inoltre non è difficile immaginare che molti concetti che vedremo sono rappresentabili come alberi (l'espressione aritmetica dell'altra volta).

La forma più comune che useremo è la rappresentazione con liste di liste (lol), la prima è la radice e poi ci sono i sotto-alberi con tutti i loro figli.

```
1 # [radice]
2 # [radice alberi]
3
4 tree = [1, [11, [111]], [12, [121], [122]], [13]]
```

La cosa comoda di python è che c'è l'assegnamento ordinato, viene particolare comodo l'unpacking iterabile, se metto l'* prima della variabile mi prendo tutto quello che resta:

```
1 a, *b = 1, 2, 3
2 print(b) #[2, 3]
```

Questo è comodo perchè verrà comodo posso prendere i figli:

```
1 root, *children = tree
```

Come già detto useremo la libreria Liblet per visualizzare gli alberi partendo dalle lol (liste di liste). È possibile fare l'unpacking anche di questi alberi con la medesima sintassi.

```
1 from liblet import Tree, side_by_side
2
3 t = Tree.from_lol(tree)
4
5 root, *children = t
```

Visite

Quando facciamo una visita possiamo scegliere tante strategie, l'unica questione è decidere quando operare sul nodo, possiamo decidere di operare all'inizio (quando lo incontro) oppure occuparmi del nodo al termine della visita ai figli. Gli ordini vengono chiamati:

- pre-ordine: visito prima il nodo e poi i figli
- post-ordine: visito prima i figli e poi il nodo
- per livello

La visita in preordine si implementa in questo modo, posto che prende la funzione che fa i suoi conti:

```

1 def preorder(tree, root):
2     root, *children = tree
3     visitor(root)
4     for child in children: preorder(child, visitor)

```

Per fare il postordine scambio lo scarico ricorsivo con la funzione che fa i calcoli, ovviamente visualizzeremo nodi in ordine diverso.

```

1 def postorder(tree, visitor):
2     root, *children = tree
3     for child in children: preorder(child, visitor)
4     visitor(root)

```

Nel caso degli alberi è possibile visitarli anche per livelli, si chiama level order. È un algoritmo che non fa uso della ricorsione, generalmente si adopera una coda in cui accumulo i figli che vedo in modo tale che mi occupo per primo di quelli che metto dentro:

```

1 def levelorder(tree, visitor):
2     Q = queue()
3
4     Q.enqueue(tree)
5     while Q:
6         tree = Q.dequeue()
7         root, *children = tree
8         visitor(root)
9         for st in children: Q.enqueue(child)

```

Alberi con attributi

Una cosa che potrebbe essere molto comoda è che potremmo avere bisogno di usare nodi con informazioni più strutturate per poter visualizzare alberi arricchiti di attributi. Per fare questo useremo un albero che abbia dict come valori e che conservi il valore numerico come valore della chiave val:

```

1 def add_attr(tree):
2     root, *children = tree
3     return [{'val': root}] + [add_attr(child) for child in children]
4
5 tree = [1, [11, [111]], [1200, [121], [122]], [13]]
6
7 add_attr(tree)

```

Gli attributi generalmente vengono calcolati, ad esempio la profondità a cui siamo. Il modo in cui si calcolano segue due direzioni:

- Attributi ereditati: calcolati per il papà e ereditati dai figli. Si usa una visita in pre-ordine.
- Attributi sintetizzati: in qualche modo otteniamo prima le informazioni sui figli e poi sintetizziamo verso l'alto (tipicamente la valutazione delle espressioni è un attributo sintetizzato). Si usa una visita in post-ordine, perchè prima devo visitare i figli.

Ad esempio vediamo come spingere in giù la profondità del nodo, (nella visita ricevo che sono figlio di un nodo profondo 4), quello che fa il visitor è aggiornare quella profondità di 1 per vedere quanto è profondo lui:

```
1 def preorder_with_value(tree, visitor, value = None):
2     root, *children = tree
3     visitor(root, value)
4     for child in children: preorder_with_value(child, visitor,
5         root['depth'])
6
7 # visitor che aggiunge l'attributo depth (pari a 1 + il valore
8 # ereditato, il caso None riguarda la radice)
9
10 def add_depth(root, value):
11     root['depth'] = value + 1 if value is not None else 0
12
13 attr_tree = add_attr(tree)
14
15 # la radice ricevera None perche e' il valore di default di value
16
17 preorder_with_value(attr_tree, add_depth)
18
19 Tree.from_lol(attr_tree)
```

Nella sintesi il codice è più semplice da scrivere perchè possiamo usare il return per ritornare il valore al padre:

```
1 def postorder_with_return(tree, visitor):
2     root, *children = tree
3     values = [postorder_with_return(child, visitor) for child in
4         children] # sara' la lista vuota se non ci sono figli
5     return visitor(root, values)
6
7 # visitor che aggiunge l'attributo max (pari al massimo tra il valore
8 # del nodo e quelli sintetizzati dai figli)
9
10 def add_max(root, values):
11     root['max'] = max([root['val']] + values)
12     return root['max']
13
14 attr_tree = add_attr(tree)
15
16 postorder_with_return(attr_tree, add_max)
17
18 Tree.from_lol(attr_tree)
```

1.2.2 Grafi

Li avremo i grafi come strumenti, meno concretamente ma avremo delle visite per passare da uno stato all'altro ma non scriveremo esplicitamente grafi. Esistono grafi non orientati e orientati. Idealmente noi useremo i grafi orientati. Tipicamente i grafi si rappresentano con:

- Matrice di adiacenza, $N \times N$ dove metto true e false dove c'è un collegamento. È $O(1)$ per il costo computazionale ma $O(n^2)$ nello spazio.

- Tengo tutte le coppie di nodi collegati se la matrice è sparsa (lista dei nodi)
- Tengo le liste di adiacenza di ogni nodo, per ogni nodo tengo in una lista tutti i nodi che ha collegati

Le rappresentazioni più comode che usiamo sono la rappresentazione con lista di archi (una tupla di tuple) oppure le liste di adiacenza rappresentata da un dict di set:

```

1  # dagli archi alla mappa delle adiacenze
2
3
4  # per ogni nodo n (sia s o t), adjacency[n] = set()
5
6  adjacency = dict()
7  for s, t in arcs:
8      adjacency[s] = set()
9      adjacency[t] = set()
10
11 # aggiungo gli outlink
12
13 for s, t in arcs: adjacency[s] |= {t}
14
15 adjacency

```

Algoritmi

La visita di un grafo è un procedimento che ci porta da un nodo ad esplorare tutti gli altri, ho in qualche modo bisogno di salvarmi da qualche parte i nodi che ho visitato e quelli che devo visitare. Immaginiamo di avere bisogno di una sacca in cui mettere le cose che abbiamo bisogno di fare (ci mettiamo dentro i figli dei nodi), dobbiamo stare attenti a non rimettere dentro quello che ci è già entrato e ha senso tirare fuori gli elementi in un certo ordine:

- FIFO: il primo nodo che metto è il primo che tolgo, i nodi sono messi in una coda. È la storia del level order. Così effettuiamo una visita in ampiezza.
- LIFO: last in first out, è una pila. Effettuiamo così una visita in profondità.

Nella visita in profondità si può dare una implementazione ricorsiva (perché la pila può essere la pila delle chiamate), quello che dobbiamo avere è una struttura dati che ci tiene quello che abbiamo già visto, per evitare di visitare di nuovo gli stessi nodi:

```

1  def depthfirst(adjacency, start, visit):
2      def walk(src):
3          visit(src)
4          seen.add(src)
5          for dst in adjacency[src]:
6              if dst not in seen:
7                  walk(dst)
8      seen = set()
9      walk(start)

```

La visita in ampiezza usa la coda, toglie dalla coda e lo visita e quando lo visita lo aggiunge:

```

1 from liblet import Queue
2
3 def breadthfirst(adjacency, start, visit):
4
5     Q = Queue()
6
7     seen = set()
8     Q.enqueue(start)
9     while Q:
10         src = Q.dequeue()
11         visit(src)
12         seen.add(src)
13         for dst in adjacency[src]:
14             if dst not in seen:
15                 Q.enqueue(dst)

```

Quello da notare nelle visite è che quello che succede nelle visite in profondità è che la pila non aumenta tanto di dimensione, l'unico problema è che se devo vedere nodi vicini è possibile che per visitarlo potrei dover fare un sacco di conti prima di arrivare a visitare quello. La visita in ampiezza d'altro canto è vero che procede per livelli, ma se il grafo è denso la coda può diventare molto grande. Quindi quello che dovremo vedere negli algoritmi di parsing è questa scelta, scendo velocemente nel grafo sperando nel successo oppure è meglio scendere per passi? quello che vedremo è che in generale se non ci sono grandi strategie l'ampiezza è l'unica strada mentre se ho qualcuno che mi dice qualcosa è meglio andare giù veloce nella strada giusta.

1.2.3 backtracking

L'idea è che possiamo decidere che alcuni rami dell'albero sono buoni e altri sono cattivi. Quindi l'idea è che se mi trovo in una strada in cui posso prevedere che se sono in un certo punto in cui non c'è speranza che andando avanti nella ricorsione non troverò mai il risultato allora posso tagliare quel ramo dell'albero. Ho una specie di soluzione parziale che posso piazzare.

Ho una soluzione candidata e se sono già capace di sapere se quella soluzione non va bene allora ritorno, se ho trovato il risultato lo stampo. Altrimenti vado avanti e faccio le chiamate ricorsive.

```

1 def backtrack(candidate):
2     if reject(candidate): return
3     if accept(candidate): output(candidate)
4     s = first(candidate)
5     while s:
6         backtrack(s)
7         s = next(candidate)

```

Come esempio vediamo come si segmentano le parole, che è un problema fondamentale nei motori di ricerca. Inanzitutto mi procuro un elenco di parole, se segmenti non è vuota e l'ultima parola del dizionario non è in WORDS butto via, altrimenti se non mi rimane niente ho trovato tutti, altrimenti provo a spaccare in due la seguente parole in tutti i modi possibili:

```

1 from urllib.request import urlopen
2
3 # WORDS sono le parole di almeno 2 caratteri (3 conta anche l'a-capo)
4
5 with
6     urlopen('https://raw.githubusercontent.com/napolux/paroleitaliane/master/paroleit
7         as url:
8     WORDS = {word.decode().strip().upper() for word in url if len(word)
9         >= 3}
10
11 print(len(WORDS))
12
13 def segmenta(segmenti, resto):
14     if segmenti and not segmenti[-1] in WORDS: return
15     if not resto:
16         print(segmenti)
17         return
18     for i in range(1, 1 + len(resto)):
19         segmenta(segmenti + [resto[:i]], resto[i:])
20
21 segmenta([], 'ILCORRIEREDELLASERAEDIZIONENOTTURNA')

```

Chapter 2

Linguaggi

2.1 Closed form

L'idea di poter definire un sotto-linguaggio per ciascuno simbolo è molto comodo perché posso partire da un linguaggio e usare quello. Il linguaggio prodotto è il prodotto dei linguaggi, la production independent è una cosa che utilizziamo molto. Le context-free consentono una struttura che viceversa perdiamo se andiamo nei linguaggi regolari ed è la questione del self embedding.

2.1.1 Self embedding

Si intende come regola ricorsiva quando nel lato destro della produzione compare il simbolo non terminale che si sta definendo:

$$A \rightarrow aAa$$

La regola ricorsiva è l'ingrediente necessario per rendere i nostri linguaggi infiniti. Questa regola ci serve ad esempio ad avere il linguaggio di Dyck, che è un linguaggio di parentesi ben formate.

$$A \rightarrow (A)$$

E questo ci porta a definire correttamente un linguaggio di programmazione. Il Context-Free è un ragionevole punto di incontro tra un linguaggio regolare e un linguaggio ricorsivo (basso ed alto livello).

2.2 Alberi di parsing

L'albero cattura la forma gerarchica di un linguaggio, è un modo per rappresentare la struttura di un linguaggio. Sia a fini linguistici sia per rappresentare la struttura di un linguaggio di programmazione.

Prima di poterci dedicare in maniera serena a questi alberi di parsing dobbiamo fare un po' di pulizia perché abbiamo delle grammatiche con difetti che vorremmo eliminare. La spazzatura che può rimanere dentro una grammatica context free è che possiamo avere dei non terminali non definiti. Cioè abbiamo messo dentro delle variabili (lettere maiuscole) che non abbiamo definito, non stanno mai a sinistra di una produzione. Questo è un problema perché non possiamo mai terminare la produzione. Sono inutili e vanno eliminati. Ci sono altre due circostanze in cui ci sono problemi, potremmo avere definito un terminale che non è mai raggiungibile da nessuna produzione, questi si chiamano simboli irraggiungibili e vanno eliminati. Infine potremmo avere una produzione ricorsiva ma in questo caso non possiamo mai avere una produzione vuota con questa variabile, questa si chiama variabile improduttiva e va eliminata. Un'altra circostanza non bella è avere un loop per derivare delle variabili.

2.2.1 Pulizia di una grammatica

1. Eliminare i non terminali non definiti
2. Eliminare i simboli irraggiungibili

3. Eliminare le variabili improduttive

4. Eliminare i loop

Quello che chiediamo è che una variabile sia derivabile non subito ma dopo un numero n di passi. Quello che chiediamo è una chiusura della funzione, dato un insieme applico f in modo ricorsivo e se questa f è chiusa significa che prima o poi arrivo ad un insieme tale per cui se applico ancora la funzione a quell'insieme rimango in quell'insieme. Un esempio di funzione chiusa è se aumento sempre gli oggetti dell'insieme ma gli oggetti fanno parte di un insieme finito.

Dentro liblet c'è un decoratore `@closure` che ci permette di fare la chiusura di una funzione. È chiaro che con una funzione di questo tipo la pulizia diventa abbastanza semplice. Vediamo un esempio di una grammatica sporca:

```
1 G = Grammar.from_string("""
2 S -> A B | D E
3 A -> a
4 B -> b C
5 C -> c
6 D -> d F
7 E -> e
8 F -> f D
9 """)
10 G
```

Le regole produttive si possono definire, in modo bottom up detrimino le produttive, diventa produttivo a sinistra quello che a destra ha tutte cose produttive:

```
1 def find_productive(G):
2
3     @closure
4     def find(prod):
5         return prod | {A for A, a in G.P if set(a) <= prod}
6
7     return find(G.T)
8
9 find_productive(G)
```

Le raggiungibili invece si possono ottenere con un processo top down, parto dal simbolo distinti e metto dentro tutti i non terminali a quali posso arrivare da qualcosa di raggiungibile:

```
1 from liblet import union_of
2
3 def find_reachable(G):
4
5     @closure
6     def find(reach):
7         return reach | union_of(set(a) for A, a in G.P if A in reach)
8
9     return find({G.S})
10
11 find_reachable(G)
```

Dopo aver definito questo posso pulire la grammatica, garantisco che tutti i simboli sono produttivi e raggiungibili:

```

1 def remove_unproductive_unreachable(G):
2     Gp = G.restrict_to(find_productive(G))
3     return Gp.restrict_to(find_reachable(Gp))
4
5 remove_unproductive_unreachable(G)

```

Attenzione che l'ordine con cui si fa questa operazione è cruciale, se eliminiamo prima i non raggiungibili e poi i non produttivi potrei avere la necessità di dover fare un'altra passata per eliminare altri non raggiungibili.

2.2.2 Dimensione degli alberi di parsing

Ha senso ragionare sulla dimensione degli alberi di parsing? sì perchè se fossero enormi non avrebbero una utilità pratica. La storia è molto semplice ed è legata al fatto che in buona sostanza la frontiera di un albero binario è lineare nel numero di nodi con N nodi abbiamo $O(N)$ foglie. Quindi quello che vogliamo dimostrare che se prendiamo una grammatica non malata un albero di derivazione non può contenere più di N nodi. Quello che facciamo è raginare bottomup, tutte le volte che vengo verso l'alto e faccio un passo agglomerativo (agglomerato con un non terminale) un nodo lo aggiungo ma almeno due ne tolgo, il che vuol dire che se questa cosa allora ho introdotto N nodi e ne ho tolti $2N$ quindi ho inserito linearmente N nodi. Le cose che mi restano da guardare è cosa succede nel caso di regole unitarie, avendo regole unitarie al massimo si va ad esplodere nella dimensione dei non terminali, perchè significa che arrivo ad un agglomeratore tramite una catena.

2.2.3 Derivazioni

Non è detto che una parola derivata abbia sempre la stessa derivazione, ci sono tante derivazioni possibili da una grammatica per la stessa parola. Le derivazioni in termini di alberi di parsing vediamo che per due derivazioni diverse abbiamo lo stesso albero di parsing ed è una situazione spiacevole perchè ci sarebbe piaciuto avere una mappa 1-1, la presenza di più derivazioni più essere più o meno critica a seconda del contesto. Ci possono essere due derivazioni che hanno due alberi di parsing diversi che è la situazione che ci preoccupa, il primo caso è facilmente risolvibile indicando delle derivazioni preferibili (nel primo caso abbiamo per finta più derivazioni perchè alla fine cambia solo l'ordine). Nel secondo caso abbiamo più alberi di parsing per più derivazioni.

```

1 # una grammatica banale per il linguaggio {a^n b^n | n > 0}
2
3 G_ab = Grammar.from_string('''
4 S -> A B
5 A -> a A | a
6 B -> b
7 ''')
8 G_ab
9
10 # due possibili derivazioni
11
12 ab_0 = Derivation(G_ab).step(
13     [(0, 0), (1, 0), (2, 1), (3, 2)]
14 )

```

```

15 ab_1 = Derivation(G_ab).step(
16     [(0, 0), (3,1), (1,0), (2,1)]
17 )
18
19 ab_0, ab_1
20
21 # ma a ben guardare lo stesso albero
22
23 side_by_side(
24     ProductionGraph(ab_0),
25     ProductionGraph(ab_1),
26 )

```

2.2.4 Dalla derivazione all'albero di parsing

Cominciamo a ragionare sul fatto che non è così ovvio il legame tra le parole, le derivazioni e gli alberi di parsing. Adesso cerchiamo di convincerci che almeno uno di questi pezzi è facilmente raggiungibile, esiste un modo semplice data una derivazione costruire un albero di parsing. Con Python posso tenermi la forma sentenziale conservando tutti gli alberi che mano a mano da questa forma sviluppo, la prima forma è il simbolo distinto e poi per ogni passo di derivazione mi dice quale pezzo della forma sentenziale va sostituito e noi sostituiamo questa con un nodo nell'albero. Sostanzialmente per ogni passaggio di derivazione sostituisco mettendo i nodi nell'albero, nella forma sentenziale tengo sempre le foglie e sopra metto da dove derivo. La seguente procedura memorizza in tree l'albero di derivazione e in frontier la sua frontiera, corrispondente alla forma sentenziale a cui è giunta la derivazione (di passo in passo) come una lista di alberi annotati.

Ciascun nodo dell'albero ha due etichette: Symbol che si riferisce ad uno dei simboli della grammatica e prord pari ad una produzione. I figli di ciascun nodo dell'albero hanno i simboli contenuti nel lato destro di prord.

Gli alberi vengono "completati" man mano che la procedura elabora i passi della derivazione; al termine le foglie degli alberi saranno simboli terminali (e prord sarà convenzionalmente definito come None).

```

1  def derivation_to_parsetree(d):
2
3      # questa variabile si riferira all'albero di derivazione di d
4      # inizialmente contiene l'albero annotato col simbolo di partenza
5      tree = Tree({'Symbol': d.G.S, 'prord': None})
6
7      # all'inizio la forma sentenziale e data da tale albero
8      frontier = [tree, ]
9
10     for nprod, pos in d.steps():
11
12         # l'albero da completare e dato dalla posizione in cui e
13         # applicata la produzione
14         curr = frontier[pos]
15
16         # risalgo dal numero alla produzione
17         prod = d.G.P[nprod]
18
19         # i figli sono dati dal lato destro d.G.P[prod].rhs

```

```

19     children = [Tree({'Symbol': X, 'prod': None}) for X in prod.rhs]
20
21     # aggiorniamo l'albero da completare
22     curr.root['prod'] = prod
23     curr.children = children
24
25     # aggiorniamo la forma sentenziale
26     frontier = frontier[:pos] + children + frontier[pos + 1:]
27
28     return tree

```

Per tornare indietro (dall'albero alla derivazione) posso fare una visita in pre-ordine arriviamo ad una derivazione left most:

```

1 def leftmost(tree):
2     return [tree.root['prod']] + [prod for child in tree.children for
3         prod in leftmost(child) if prod]
4
5 lm_0 = leftmost(pt_0)
6 lm_0

```

Da notare che se facciamo una visita in post-ordine non otteniamo una right most, ma una right most al contrario. Questo tipo di ambiguità è ineliminabile. Questo è logicamente un problema quando abbiamo operatori non associativi (es. sottrazione e divisione).

Quello che dovremo fare quando scriveremo una grammatica:

1. Non è sempre possibile
2. Non è automatico
3. Introduco N non "semantici": è chiaro che poi questi N andranno eliminati perchè producono alberi di parsing pieni di mondezze

2.2.5 Possibili soluzioni

La ricorsione a sinistra determina un'associatività a sinistra mentre se è a destra si usa una ricorsione a destra. Quello che facciamo è inserire dei simboli in più nel linguaggio per gestire tutti i casi:

```

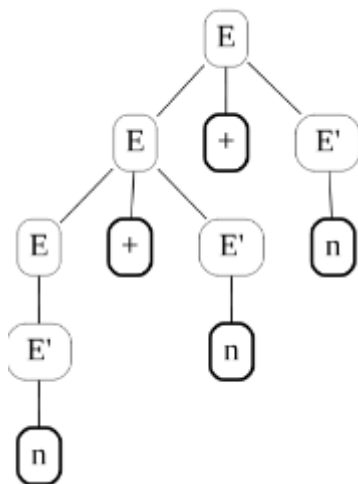
1 G_la = Grammar.from_string("""
2 E -> E + E' | E'
3 E' -> n
4 """)
5 G_la

```

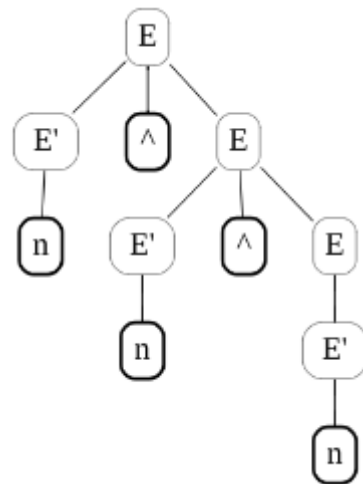
```

1 G_ra = Grammar.from_string("""
2 E -> E' ^ E | E'
3 E' -> n
4 """)
5 G_ra

```



(a) Associatività Sinistra



(b) Associatività Destra

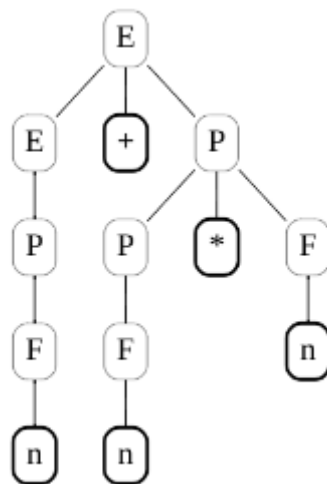
Figure 2.1: Confronto tra associatività sinistra e destra

Quando ho la menata della precedenza, operatori binari con precedenza diversa, questo si traduce con l'introduzione di simboli ulteriori abbiamo ora n . Questi simboli sono terminali che rappresentano la precedenza degli operatori.

```

1  G_p = Grammar.from_string("""
2  E -> E + P | P
3  P -> P * F | F
4  F -> n
5  """)

```



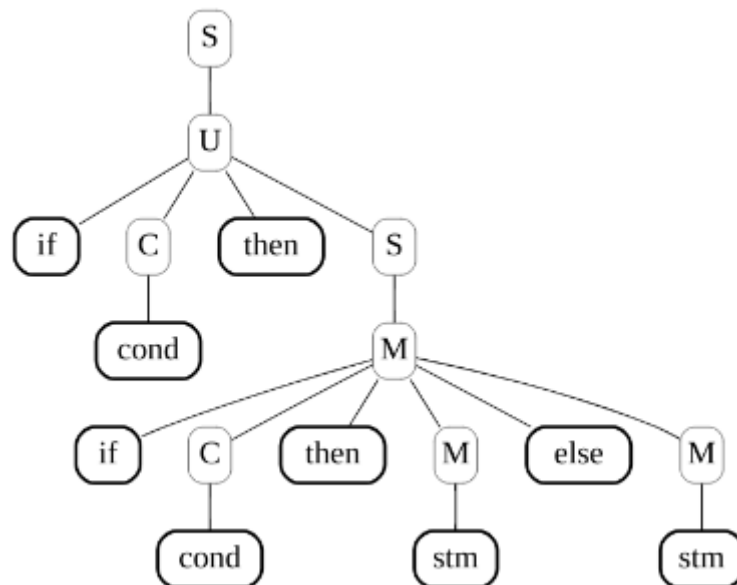
Nel caso del dangling else (if else if) diventa una cosa ancora più dolorosa c'è un'interplate tra SMUC che è difficile da ricordare risolve la cosa perché tiene l'else legato all'if più vicino.

```

1  G_if = Grammar.from_string("""
2  S -> M | U
3  M -> if C then M else M | stm
4  U -> if C then M else U | if C then S

```

```
5 C -> cond
6 """)
```



Il punto cruciale è che context free siamo al giusto livello (non ci siamo persi le parentesi sotto) abbiamo alberi di parsing lineari nella lunghezza della parola, ma dobbiamo stare attenti che avremo ambiguità ed in questo caso dobbiamo o modificare la grammatica che però sarà piena di non terminali e spesso questa procedura passa da trucchi e non c'è niente di teorico (che si possono copiare o inventare).

Chapter 3

Parsing

Quello che vogliamo fare ora è passare partendo da un simbolo distinto espandendo una serie di regole per cui alla fine del processo avrò la parola. L'altro processo è l'opposto parto dalla parola e cerco di capire quali sono gli ultimi pezzi che hanno composto la parola e poi proceso all'indietro fino ad arrivare al simbolo distinto, in questo modo cerco di riaggregare i pezzi andando dal basso verso l'alto. Questi a grandi linee sono le due strategie che applicheremo per il parsing. I due modi si chiamano:

1. **Top-down:** Parto dal simbolo distinto e cerco di arrivare alla parola
2. **Bottom-up:** Parto dalla parola e cerco di arrivare al simbolo distinto

Notiamo che questi approcci possono essere applicati a tutte le grammatiche, ad esempio vediamo una grammatica context sensitive, nel caso top down:

```

1 w = 'aabbcc'
2
3 G = Grammar.from_string("""
4 S -> a S Q | a b c
5 b Q c -> b b c c
6 c Q -> Q c
7 """, False)
8
9 steps = (0, 0), (1, 1), (3, 3), (2, 2)
10
11 for nprod, pos in steps: print(pos, G.P[nprod])

```

Nel caso bottom up quello che a volte conviene fare è ribaltare la grammatica, le produzioni left producono le right, questo ovviamente va a rovinare la nostra grammatica, poi possiamo mettere anche un simbolo Inizio e Fine per indicare l'inizio e la fine della produzione. Spesso lavorando in questo modo si produce una derivazione right-most (sostituisco il simbolo più a destra della derivazione non sentenziale e sostanzialmente faccio i passi da destra a sinistra dall'ultimo al primo). Posso decidere dall'alto in basso e questo produce derivazioni left-most oppure posso andare dal basso verso l'alto cercando di aggregare e ottengo derivazioni right-most ma che devo leggere dal basso verso l'alto.

```

1 GR = Grammar.from_string("""
2 Inizio -> a a b b c c
3 a S Q -> S
4 a b c -> S
5 b b c c -> b Q c
6 Q c -> c Q
7 S -> Fine
8 """, False)
9
10 steps = (0, 0), (3, 2), (4, 3), (2, 1), (1, 0), (5, 0)
11
12 for nprod, pos in steps: print(pos, GR.P[nprod])

```

3.1 NPDA

In realtà questo tipo di comportamento viene modellato nell'ambito del parsing attraverso la nozione di automa che ha bisogno di un nastro di input ed un area di memoria per

tenere il processo di parsing, sostanzialmente deve fare la scansione della parola e mano a mano costruire l'albero di derivazione. Tutti gli algoritmi che vedremo hanno bisogno di una pila (stack) come forma di memoria per tenere traccia del processo di parsing perché il tipo di lavoro che fanno non richiede un accesso casuale alla memoria. Ma come fanno a decidere questi automi? hanno un dispositivo di controllo che è in grado di determinare cosa fare, come spostarsi sul nastro e cosa scrivere e prendere dalla pila. La cosa più intuitiva che uno può fare è avere una mente onnisciente che è in grado di determinare cosa l'automata in ogni situazione deve fare. Quello che dovremo fare nel nostro lavoro è costruire questo controllo e poi trovare un modo per eliminare il non determinismo perché non avremo l'oracolo che sa tutto.

Quello che accadrà è che data una descrizione della grammatica sarà possibile realizzare la struttura di controllo dell'automata in maniera automatica, potremmo pensare ad un programma che dato in pasto la grammatica G produca l'automata, questi programmi si chiamano parser generator. Un'altro modo molto usuale, che descrive una grande famiglia di algoritmi di parsing, è quello di avere un controllo universale (che funziona per ogni grammatica) trasformando una tabella che rappresenta le informazioni che G contiene in una forma che fa in modo che il controllo sappia cosa fare, questo metodo si chiama table driven.

Nell'analizzare il comportamento di questi parser ci sono due grandezze che vogliamo vedere:

1. Lo spazio in memoria, quanto spazio consuma l'automata nel funzionamento sempre in rapporto alla lunghezza della parola
2. Il tempo di esecuzione, quanto tempo impiega l'automata a processare la parola

Evidente più è alto il tipo di grammatica più è alta la memoria e più scendiamo il contrario. Noi ci occuperemo delle context-free e per quel che concerne gli altri due livelli della gerarchia osserviamo questo:

- Le tipo 0 (unrestricted) rappresentano gli insiemi ricorsivamente enumerabili, quindi chiedersi se una parola appartiene ad una grammatica di tipo 0 equivale a chiedersi se il programma termina, problema indecidibile
- Le tipo 1 (context sensitive) sono più deboli delle unrestricted, sono gli insiemi accettati da una macchina di Turing non deterministica, problema decidibile ma non in tempo polinomiale bensì in tempo e spazio esponenziale. Questo è il livello in cui si colloca il parsing naturale (linguaggi naturali).
- Le tipo 2 (context free) sono gli insiemi accettati da un NPDA, problema decidibile in tempo polinomiale.

Una cosa che vale la pena considerare nell'ambito del parsing context-free esiste una possibile divisione del lavoro che dobbiamo fare raggruppando sulla base di qualche criterio. Una prima grande dicotomia è se gli algoritmi funzionano in maniera:

1. top-down
2. bottom-up

Un'altra dicotomia piuttosto interessante è il modo in cui l'input viene analizzato, perchè l'automa può andare avanti ed indietro nell'input:

1. Parser direzionale: L'automa va avanti e indietro nell'input, in ordine, se lo ciuccia man mano che riceve i byte
2. Parse non direzionale: il parser può adoperare delle porzioni del nastro diverse, ad esempio nell'uso degli editor abbiamo la colorizzazione della sintassi, in questo caso il parser non è direzionale perchè se fosse direzionale dovrei ricostruire tutto l'albero per ogni modifica.

Un'altra possibile dicotomia che abbiamo già in qualche modo accennato è il fatto che purtroppo abbiamo il non determinismo e per risolvere questo potrebbe provare tutti i passi agendo:

1. Depth first
2. Breadth first

Una possibilità cruciale per eliminare il problema del non determinismo è avere un parsing che a priori non accetti qualsiasi grammatica ma solo quelle che sono in una certa forma, quelle deterministiche (si rispippola la grammatica in modo che sia deterministica). Questo ci crea dei parsing molto meno potenti.

Cosa vedremo noi:

1. Non directional methods Bottom-up: CYK parser
2. Deterministic directional:
 - (a) Top-down: LL parser, mescolare parsing e traduzione è più semplice dell'LR. Sono meno efficienti ma pace, noi useremo un parser generator LL(*). Noi vedremo LL(1).
 - (b) Bottom-up: LR parser, vedremo LR(0)

3.2 CYK parser

L'algoritmo CYK si basa su una tabella che rappresenta la comprensione temporanea che ha l'algoritmo del processo di parsing (della costruzione dell'albero di parsing) e questa tabella può essere facilmente riempita nel caso in cui la grammatica abbia una forma semplificata che per il momento noi assumeremo. Le regole che vorremo avranno solo questa forma, una produzione non terminale o di una coppia:

1	A -> BC
2	A -> a

La tabella di questo algoritmo è fatta così, sulla base abbiamo la parola e poi ha una serie di celle in cui in ciascuna cella contiene l'informazione della sotto-parola la cui lunghezza è data dalla riga in cui mi trovo e che raccontano l'idea che si è fatto l'algoritmo di parsing della stringa lunga due a partire dalla posizione della tabella sotto. Nella posizione i, l rappresenta l'informazione che ho sul parsing del prefisso della parola che comincia lì ed è lunga l :

```

1 INPUT = 'unaprova'
2
3 n = len(INPUT)
4
5 R = CYKTable()
6 for l in range(1, n + 1):
7     for i in range(1, n - l + 2):
8         R[i, l] = INPUT[(i) - 1: (i + l) - 1]

```

unaprova							
unaprov	naprova						
unapro	naprov	aprova					
unapr	napro	apro	prova				
unap	napr	apro	prov	rova			
una	nap	apr	pro	rov	ova		
un	na	ap	pr	ro	ov	va	
u	n	a	p	r	o	v	a

Osserviamo che la tabella può essere riempita in due modi, siccome ogni posizione riguarda un sottoinsieme la posso riempire in due modi:

1. offline: parto dal fondo a sinistra e comincio a mettere le lettere singole, poi sopra le coppie e così via. Offline perchè per riempirla devo aver visto tutto l'input
2. online: la riempio in diagonale partendo dal basso a sinistra, mano a mano che vedo caratteri posso riempire sempre più celle

```

1 def offline(fill, n):
2     R = CYKTable()
3     for l in range(1, n + 1):
4         for i in range(1, n - l + 2):
5             R[i, l] = fill(R, i, l)
6     return R
7
8 def online(fill, n):
9     R = CYKTable()
10    for d in range(1, n + 1):
11        for i in range(d, 0, -1):
12            R[i, d - i + 1] = fill(R, i, d - i + 1)
13    return R

```

Noi ovviamente non la riempiamo di parti di parola ma di produzioni, per filtrare le produzioni useremo la funzione di python filter che filtra cose in base ad una funzione che gli passiamo. Quando siamo in alto nella tabella dobbiamo mettere una produzione che da A produce BC, perchè vogliamo metterci qualcosa che produce quello che c'è sotto

quindi sotto dobbiamo andare a vedere se B produce un pezzo di quello che c'è sotto e la C il resto.

In buona sostanza se sono a lunghezza 1 ci metto tutti i terminali dove A produce a dove a è proprio la parola (lettera negli esempi) che sto cercando, se sono a lunghezza 2 metto tutte le produzioni che producono due terminali B e C dove B produce la parola da i a k e c da k in poi.

```

1 def cyk_fill(G, INPUT):
2     def fill(R, i, l):
3         res = set()
4         if l == 1:
5             for A, (a,) in filter(Production.such_that(rhs_len = 1), G.P):
6                 if a == INPUT[i - 1]: res.add(A)
7         else:
8             for k in range(1, l):
9                 for A, (B, C) in filter(Production.such_that(rhs_len = 2), G.P):
10                    if B in R[i, k] and C in R[i + k, l - k]: res.add(A)
11         return res
12     return fill

```

Vediamo degli esempi su una grammatica a^n :

```

1 G = Grammar.from_string("""
2 S -> A S
3 A -> a
4 S -> .
5 """)
6
7 INPUT = 'aaa.'
8
9 online(cyk_fill(G, INPUT), len(INPUT))

```



```

1 INPUT = 'aa.a.'
2
3 online(cyk_fill(G, INPUT), len(INPUT))

```

```

1 # fig. 4.15, pag. 123
2
3 G = Grammar.from_string("""
4 Number -> 0|1|2|3|4|5|6|7|8|9
5 Number -> Integer Digit
6 Number -> N1 Scale' | Integer Fraction
7 N1 -> Integer Fraction
8 Integer -> 0|1|2|3|4|5|6|7|8|9
9 Integer -> Integer Digit
10 Fraction -> T1 Integer

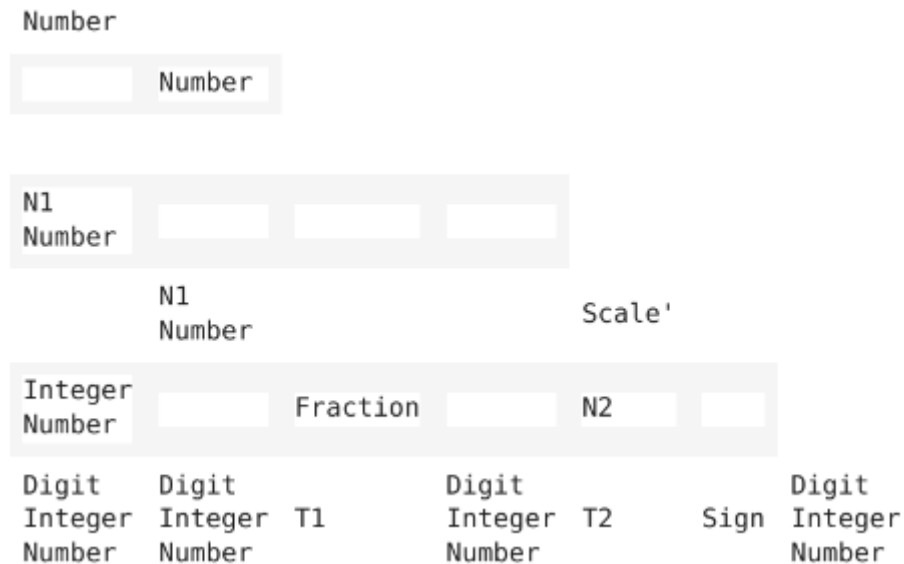
```



```

11 T1 -> .
12 Scale' -> N2 Integer
13 N2 -> T2 Sign
14 T2 -> e
15 Digit -> 0|1|2|3|4|5|6|7|8|9
16 Sign -> + | -
17 """)

```



Negli esempi si vede bene che sotto mettiamo tutti i terminali delle produzioni a sinistra che producono la parola o lettera (ad esempio A produce a quindi ci metto A per tutte le volte che ci sono a nella parola). Poi per le celle sopra ci chiediamo se c'è una produzione che produce $A \rightarrow BC$ dove B produce la sotto parola da i a k e C da k in poi (ad esempio $S \rightarrow A, S$).

Per guardare se la parola fa parte del linguaggio basta guardare la cella in alto della tabella, se è riempita significa che è ok.

3.2.1 Albero di parsing

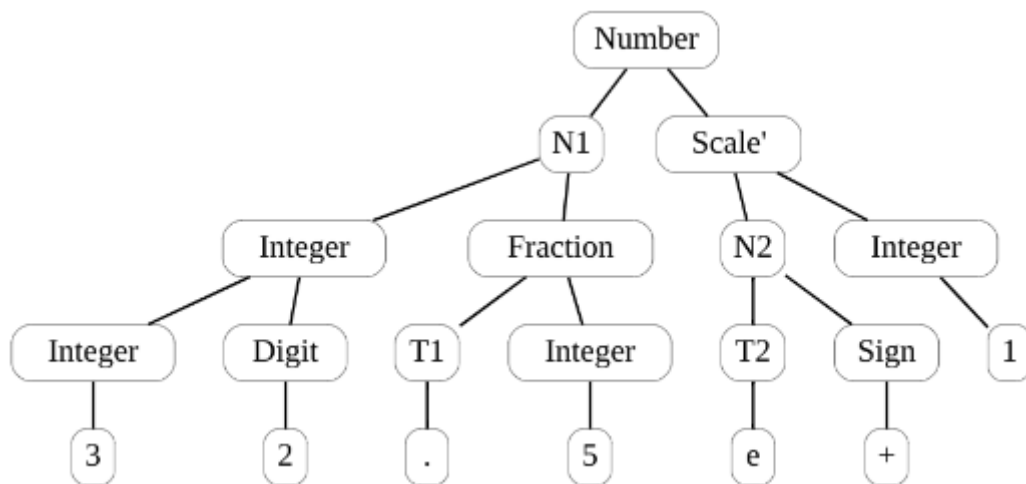
Dobbiamo ricostruire l'albero di parsing da questa tabella, al momento ci accontentiamo di costuire l'albero i cui nodi sono i non terminali e gli archi le produzioni (non ha dentro le produzioni). Non è super difficile perchè possiamo approcciare questa cosa ricorsivamente. Scriviamo una funzione ricorsiva `fake_parse` che (usando la tabella R, la

grammatica G e l'input INPUT) dato un non terminale, il punto d'inizio e la lunghezza, restituisca l'albero di parsing radicato in quel non terminale e che deriva la sottostringa specificata.

```

1 def cyk_fill(G, INPUT):
2     def fill(R, i, l):
3         res = set()
4         if l == 1:
5             for A, (a,) in filter(Production.such_that(rhs_len = 1), G.P):
6                 if a == INPUT[i - 1]: res.add(A)
7         else:
8             for k in range(1, l):
9                 for A, (B, C) in filter(Production.such_that(rhs_len = 2), G.P):
10                    if B in R[i, k] and C in R[i + k, l - k]: res.add(A)
11     return res
12     return fill

```



3.2.2 Riduzione in forma normale di Chomsky

La forma normale di Chomsky è una forma normale in cui tutte le produzioni sono della forma:

```

1 A -> BC
2 A -> a

```

Per ottenere una grammatica in forma normale di Chomsky dobbiamo fare due passaggi:

1. Eliminare le regole unitarie
2. Eliminare le produzioni più lunghe di 2, riduzione in forma normale, trasformazioni produzione non solitarie
3. Eliminare le epsilon regole

```

1 def remove_unproductive_unreachable(G):
2     def find_productive(G):
3         @closure
4         def find(prod):
5             return prod | {A for A, a in G.P if set(a) <= prod}
6         return find(G.T)
7     def find_reachable(G):
8         @closure
9         def find(reach):
10            return reach | union_of(set(a) for A, a in G.P if A in reach)
11        return find({G.S})
12    Gp = G.restrict_to(find_productive(G))
13    return Gp.restrict_to(find_reachable(Gp))
14
15 def cyk(G, INPUT):
16     def fill(R, i, l):
17         res = set()
18         if l == 1:
19             for A, (a,) in filter(Production.such_that(rhs_len = 1), G.P):
20                 if a == INPUT[i - 1]: res.add(A)
21         else:
22             for k in range(1, l):
23                 for A, (B, C) in filter(Production.such_that(rhs_len = 2), G.P):
24                     if B in R[i, k] and C in R[i + k, l - k]: res.add(A)
25         return res
26     R = CYKTable()
27     for l in range(1, len(INPUT) + 1):
28         for i in range(1, len(INPUT) - l + 2):
29             R[i, l] = fill(R, i, l)
30     return R

```

3.2.3 Eliminazione epsilon-regole

Una epsilon regola è una regola della forma $A \rightarrow \epsilon$, cioè una regola che produce la parola vuota. Per eliminare una epsilon regola poteri duplicare la regola, cioè se ho una regola $A \rightarrow \epsilon$ e una produzione $B \rightarrow \alpha A B$ posso da qui crearne due con A_1 e A_2 . Non sempre è così comodo. La prima osservazione che facciamo è che abbiamo trasformato G in g primo in cui non abbiamo le epsilon regole ma il costa è che per ogni epsilon regola che eliminiamo produce 2^n produzioni (dove n è la lunghezza della regola). Quindi sappiamo che $|G'| \geq 2|G|$. Sostanzialmente con due passi ottenuti tramite chiusura posso rimpiazzarte un simbolo nei lati destri con `replace_in_rhs` e quindi applicare il primo passo a tutti i simboli che compaiono in una epsilon regola con `inline_epsilon_rules`.

La prima cosa che dobbiamo fare è il rimpiazzamento di A con A_1 in tutte le produzioni, poi prendo tutte le produzioni che contengono A e cancello o sostituisco con A primo le occorrenze di A , se invece on l'ho beccato in B lascio così com'è:

```

1 @closure
2 def replace_in_rhs(G, A):
3     Ap = A + ''
4     prods = set()
5     for B, Beta in G.P:
6         if A in Beta:

```

```

7     pos = Beta.index(A)
8     prods.add(Production(B, Beta[:pos] + Beta[pos + 1:]))
9     prods.add(Production(B, Beta[:pos] + (Ap, ) + Beta[pos + 1:]))
10    else:
11        prods.add(Production(B, Beta))
12    return Grammar(G.N | {Ap}, G.T, prods, G.S)
13
14    # esempio d'uso
15
16    U = Grammar.from_string("""
17    S -> x A y A z
18    A -> a
19    """)
20    replace_in_rhs(U, 'A').P

```

A $a_{(0)}$

S $x y z_{(1)} \mid x y A' z_{(2)} \mid x A' y z_{(3)} \mid x A' y A' z_{(4)}$

A questo punto per sistamarle dobbiamo fare una chiusura, perchè abbiamo prodotto delle epsilon regole, prende tutti i non terminali che non abbiamo ancora visto e se quel terminale tra i lati destri di A c'è una epsilon regola faccio il rimpiazzamento e segno che l'ho fatto.

```

1  @closure
2  def inline_epsilon_rules(G_seen):
3      G, seen = G_seen
4      for A in G.N - seen:
5          if (epsilon, ) in G.alternatives(A):
6              return replace_in_rhs(G, A), seen | {A}
7      return G, seen
8
9      # esempio d'uso
10
11      U = Grammar.from_string("""
12      S -> A
13      A -> B C
14      B -> epsilon
15      C -> epsilon
16      """)
17      U, _ = inline_epsilon_rules((U, set()))
18
19      U.P

```

A $\epsilon_{(0)} \mid B'_{(1)} \mid C'_{(5)} \mid B' C'_{(6)}$

C $\epsilon_{(2)}$

S $\epsilon_{(3)} \mid A'_{(7)}$

B $\epsilon_{(4)}$

Osserviamo che questo procedimento non ha tolto le epsilon regole ma le ha messe inline, le mette dove accadevano, quello che succede è che queste regole prima o poi diventano unreachable quindi prima o poi questa regola la toglieremo. L'altro effetto è che solleva la epsilon fino al simbolo distinto, quindi quello che accade è che questo linguaggio potrebbe produrre la parola vuota. Usando i passi precedenti è semplice arrivare al passo di eliminazione:

```

1 def eliminate_epsilon_rules(G):
2     Gp, _ = inline_epsilon_rules((G, set()))
3     prods = set(Gp.P)
4     for Ap in Gp.N - G.N:
5         A = Ap[:-1]
6         for alpha in set(Gp.alternatives(A)) - {(epsilon, )}:
7             prods.add(Production(Ap, alpha))
8     return Grammar(Gp.N, Gp.T, prods, Gp.S)
9
10 # esempio d'uso (fig. 4.10, pag. 120)
11
12 U = Grammar.from_string("""
13 S -> L a M
14 L -> L M
15 L -> epsilon
16 M -> M M
17 M -> epsilon
18 """)
19
20 eliminate_epsilon_rules(U).P

```

$$\begin{array}{l}
 M \quad \epsilon_{(0)} \mid M'_{(6)} \mid M' M'_{(14)} \\
 L \quad L' M'_{(1)} \mid M'_{(10)} \mid \epsilon_{(11)} \mid L'_{(12)} \\
 M' \quad M'_{(2)} \mid M' M'_{(4)} \\
 L' \quad L'_{(3)} \mid L' M'_{(8)} \mid M'_{(15)} \\
 S \quad a M'_{(5)} \mid a_{(7)} \mid L' a_{(9)} \mid L' a M'_{(13)}
 \end{array}$$

3.2.4 Eliminazione delle unit rules

Le regole unitarie sono quelle della forma $A \rightarrow B$, cioè una variabile che deriva un'altra variabile. In questo caso mi aspetto anche che ci sia una C che deriva B e A che deriva qualcos'altro che non è B . Quello che si fa è in tutte le regole dove c'è la A ci metto tutte le alternative. Dato $B \rightarrow \omega_1 \mid \omega_2$ e $C \rightarrow \alpha A B$ posso fare $C \rightarrow \alpha \omega_1 \mid \alpha \omega_2$. Faccio questa sostituzione a meno che non mi trovi nel caso del loop $B \rightarrow B$. Prendo tutte le produzioni che sono lunghe 1, le spacco, dico che A produce B , se B è un non terminale prendo le produzioni e faccio l'inline, faccio in modo che A produca tutte le produzioni di B e poi elimino A produce B :

```

1 def eliminate_unit_rules(G):

```

```

2  @closure
3  def eliminate(G_seen):
4      G, seen = G_seen
5      for P in set(filter(Production.such_that(rhs_len = 1), G.P)) - seen:
6          A, (B, ) = P
7          if B in G.N:
8              prods = (set(G.P) | {Production(A, alpha) for alpha in
9                          G.alternatives(B)}) - {P}
10             return Grammar(G.N, G.T, prods, G.S), seen | {P}
11         return G, seen
12     return eliminate((G, set()))[0]
13
14 # esempio d'uso
15 U = Grammar.from_string("""
16 S -> A
17 A -> B
18 B -> A | b
19 """)
20
21 eliminate_unit_rules(U).P

```

Notiamo che questo porta ad avere una grammatica con delle cose irraggiungibili (dopo che eliminiamo le epsilon regole e i non unitari). Se voglio fare pulizia uso la tecnica dell'altra volta, applico il metodo `remove_unproductive_unreachable`, ma ancora non è quello da cui siamo partiti perchè ad esempio ci sono dei non terminali oppure delle cose lunghe 3. Ci rimangono due passaggi:

1. Eliminare i non solitari: $A \rightarrow \alpha\beta$. dove alpha e beta non sono terminali
2. Eliminare le produzioni più lunghe di 2

3.2.5 Eliminazione i non solitari

Un solitario è un simbolo non terminale se esiste almeno una derivazione in cui appare, contribuendo alla produzione di stringhe terminali. Per ogni solitario che trovo in giro produco una regola unitaria lunga 1 e sostituisco secco, non ho bisogno neanche di fare le chiusure, cerco le produzioni $A \rightarrow B$ e cerco nel lato destro e guardo cosa sono, se sono dei non terminali li lascio come sono, se sono dei terminali li sostituisco con N e la letterina, dopo di che sostituisco la produzione $Na \rightarrow a$ ad esempio:

```

1  def transform_nonsolitary(G):
2      prods = set()
3      for A, alpha in G.P:
4          prods.add(Production(A, [f'N{x}' if x in G.T else x for x in alpha]
5                                if len(alpha) > 1 else alpha))
6          prods |= {Production(f'N{x}', (x, )) for x in alpha if x in G.T and
7                        len(alpha) > 1}
8      return Grammar(G.N | {A for A, alpha in prods}, G.T, prods, G.S)
9
10 U = Grammar.from_string("""
11 S -> x S y S x
12 """)

```

```

12 transform_nonsolitary(U).P
13
14 Ny Y(0)
15 Nx X(1)
16 S Nx S Ny S Nx

```

Non e' sempre cosi' facile, per le produzioni lunghe faccio produrre ad un primo pezzo A1, x1, x2 poi ad un secondo pezzo A2 faccio produrre A1 x2, così via fino in fondo dove avrò A produce A3 x7, partendo da una produzione $A \rightarrow x_1x_2x_3x_4x_5x_6x_7$:

```

1 def make_binary(G):
2     prods = set()
3     for A, alpha in G.P:
4         if len(alpha) > 2:
5             Ai = f'{{A}}{1}'
6             prods.add(Production(Ai, alpha[:2]))
7             for i, Xi in enumerate(alpha[2:-1], 2):
8                 prods.add(Production(f'{{A}}{i}', (Ai, Xi)))
9                 Ai = f'{{A}}{i}'
10            prods.add(Production(A, (Ai, alpha[-1])))
11        else:
12            prods.add(Production(A, alpha))
13    return Grammar(G.N | {A for A, alpha in prods}, G.T, prods, G.S)

```

Ora dobbiamo ricostruire l'albero di parsing, ci verrà più facile rispetto a fare una trasformazione di alberi sostituendo la tabella CYK qualcos'altro.

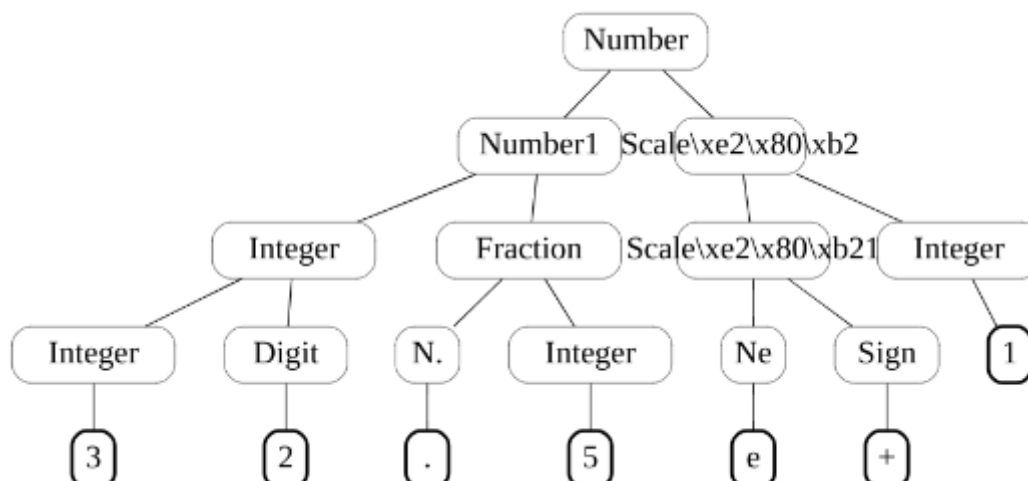
3.2.6 Costruzione albero parsing left-most

Dato un certo non terminale ci diamo come obiettivo costruire un fattore di una parola di input che partiva da una certa posizione con una certa lunghezza. Se la lunghezza è 1 devo andare a cercare una produzione della forma $X \rightarrow i-1$, questa cosa funziona perchè la tabella è costruita correttamente. Se andiamo a riprenderlo è uguale a fake parse, non metto gli alberi ma metto le produzioni. Quindi per ottenere una derivazione leftmost ragioniamo come per la funzione fake_parse ma invece di restituire un albero restituiamo l'indice della produzione in gioco:

```

1 def get_leftmost_prods(G, R, INPUT):
2     def prods(X, i, l):
3         if l == 1:
4             return [G.P.index(Production(X, (INPUT[i - 1],)))]
5         for A, (B, C) in filter(Production.such_that(lhs = X, rhs_len = 2),
6                                G.P):
7             for k in range(1, l):
8                 if B in R[i, k] and C in R[i + k, l - k]:
9                     return [G.P.index(Production(A, (B, C)))] + prods(B, i, k) +
10                        prods(C, i + k, l - k)
11     return prods(G.S, 1, len(INPUT))
12
13 leftmost_prods = get_leftmost_prods(G_cnf, R, INPUT)
14 leftmost_prods
15 [28, 31, 9, 12, 7, 37, 20, 11, 1, 6, 32, 15, 30]
16 d = Derivation(G_cnf).leftmost(leftmost_prods)
17 ProductionGraph(d)

```



Bisogna considerare anche cosa succede nelle produzioni non binarie, mi piacerebbe avere una funzione `derives` che prende una sequenza di simboli, una `i` e una `l` e restituisce una serie di lunghezze `ln` tale per cui `l0` è la lunghezza derivata da `w0` input `i + i + l0`. Per farlo dovrà fare una ricorsione sulla tabella. Mi sono perso mannaggia a me, ma era abbastanza inseguibile, prova a vedere dal libro.

3.3 Parsing Top-down, caso generale

Andiamo dal basso verso l'alto, da sinistra a destra senza occuparci particolarmente delle grammatiche. Cominciamo ragionando sull'automa, abbiamo un nastro che legge da sinistra verso destra e teniamo una pila in cui manteniamo lo stato del parsing e poi il controllo che facendo uso del nastro che legge in modo direzionale e della pila deve riuscire a risolvere il problema del riconoscimento, anzi meglio dovrebbe sputare l'albero di parsing della parola.

Facciamo un'assunzione sulla forma della grammatica che ci viene comoda per definire il controllo, una volta definito il controllo riusciremo a rilassare questa assunzione.

Proviamo a ragionare su questa grammatica:

```

1  S -> aBC
2  A -> aB | b
3  C -> a
4
5  w = aaba
  
```

Il riconoscimento è molto semplice se ci basiamo sulle produzioni, partendo da `S` vediamo che produce `a`, ma cosa ci facciamo della `B` e della `C`? adesso il punto cruciale dato che voglio una left-most mi occupo di `B`, che a questo punto posso solo sostituire con la sua produzione ottenendo `aB`, a questo punto sempre dato che voglio una left-most devo occuparmi della nuova `B` generata e non ho scelta se non sostituirla con `aB`, vado avanti così fino alla fine.

L'idea è che inizio con una pila sulla quale ho il simbolo di partenza `S`, a questo punto applico la regola 0 e cancello la `S` e metto sulla pila in ordine inverso le produzioni che ho appena applicato (ci metto solo i non terminali). A questo punto la pila contiene `CB`,

PILA	HEAD	STACK
S	a	BC
B	a	B
B	b	B
C	a	B

Table 3.1: Tabella di controllo del parser top-down

tolgo la B dalla cima della pila, con la sua produzione mi mangio la seconda a e metto sulla pila la B che resta dalla produzione di B, vado avanti così fino a che non ho più nulla da mangiare (mangiare sono i terminali minuscoli che vogliamo trovare che compongono la parola) e la pila è vuota.

Il controllo che abbiamo possiamo considerarlo come una tabella che ha il top della pila, la head della parola e lo stack che contiene i non terminali che sono stati messi in pila:

Mi voglio tenere anche la storia della derivazioni. Nella funzione della libreria mettiamo un # alla fine della pila e della parola per sapere quando fermarci, ci fermiamo quando arriviamo al cancelletto della pila e al cancelletto della parola. Questa descrizione istantanea può evolvere attraverso un passo di predizione, ho deciso che produzione applicare e quindi il passo di predizione ci porta ad avere tolto il lato sinistro della produzione dalla pila e messo il lato destro della produzione sulla pila. Posso semplificare applicando due nuove regole:

1. $(x, x) \rightarrow //$ passo di predizione
2. $(A,) \rightarrow \chi \in N$ passo di Match

Quindi a questo punto il controllo o mette una cosa sulla pila oppure toglie dalla pila se sulla cima c'è un non terminale che sta anche sul nastro. Sostanzialmente la testina legge le lettere della parola, il controllo se trova sulla cima della pila un terminale (lettera minuscola) lo consuma e lo toglie dalla pila spostando la testina del nastro a destra, se trova un non terminale lo sostituisce con la produzione e lo mette sulla pila. Se la cima della pila è il cancelletto e la testina è sul cancelletto allora ho finito.

In questo esempio vediamo una simulazione su visite del DAG delle computazioni, in ogni nodo i conserviamo la descrizione istantanea più la derivazione che ha condotto a tale derivazione (la grammatica e' quella di prima). Notiamo che all'inizio la pila viene mostrata da sinistra a destra, la cima della pila e' il carattere più a sinistra e che sulla pila all'inizio e' stato aggiunto il simbolo terminale e poi la S e in coda al nastro e' stato aggiunto il simbolo terminale mentre tra parentesi abbiamo il passo di derivazione che ha portato a quella situazione.:

```

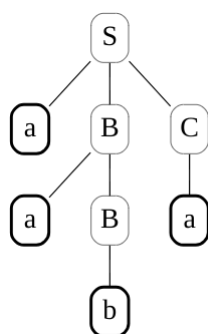
1 i = TopDownInstantaneousDescription(G, 'aaba')
2
3 output: (), S#, |aaba#
4
5 i = i.predict(G.P[0])
6 output: (s -> a B C,), aBC#, |aaba#
7
8 i = i.match()
```

```

9 output: (S -> a B C,), BC#, a | aba#
10
11 i = i.predict(G.P[2])
12 output: (S -> a B C, B -> a B, B -> b), bC#, aa|ba#
13
14 .
15 .
16 .
17
18 i = i.match()
19 output: (S -> a B C, B -> a B, B -> b, C -> a), #, aaba|#

```

Notiamo che i passi raccolti (nello stesso ordine di quello con cui sono usati da predict) sono quelli di una derivazione leftmost:



3.3.1 Funzione stato prossimo

Per ora abbiamo visto una grammatica in cui la parte di sinistra non aveva ripetizioni, normalmente potremmo avere una grammatica in cui:

```

1 S -> aB | aC
2 (S, a) -> aB
3 (S, a) -> aC

```

E questo produce una situazione di non determinismo. Per risolvere questo problema esistono tecniche teoriche e pratiche. La tecnica teorica è quella di avere un dio che dice ad ogni passo cosa fare, a noi non basta perchè non abbiamo dio direttamente da interrogare. Possiamo immaginarci un grafo della composizione istantanea in cui i nodi sono le istantanee sulla pila e gli archi sono i passi di predizione e match. Nel caso di non determinismo abbiamo più archi che partono dallo stesso nodo (perchè sono tutte le possibili produzioni che posso applicare). Questa la chiamiamo funzione a stato prossimo, quindi partendo dalla pila e dal nastro (con la parola) possiamo costruire questo grafo che poi possiamo visitare. In questo grafo troviamo dei nodi morti (dead end) che non portano a nulla e dei nodi che portano a qualcosa. Devo trovare una tecnica di visita che mi porti a trovare i nodi che portano a qualcosa (verdi), la prima idea è una visita in ampiezza in cui posso anche fermarmi quando trovo il primo nodo verde. Il secondo aspetto è che potrei avere delle situazioni in cui arrivo allo stesso nodo ma in modi diversi, sempre il problema dell'ambiguità. Ad esempio in questa grammatica:

```

1 S -> A | B

```

Quello che faccio è considerare uguali due istantanee se la pila e la testina sono uguali (non mi interessa come arrivo a quella situazione), $i = i'PILA;TESTINA$.

Cominciamo ad immaginare come produrre questo grafo, concentriamoci sulla visita in ampiezza. La cosa più facile è costruire una funzione stato prossimo, voglio costruire una funzione che data una situazione istantanea mi restituisca tutti gli archi. Mi tengo insieme a questi archi un'etichetta che mi dice se questo arco deriva da un passo di produzione o di match.

```

1 # restituisce un elenco di coppie (s, tid)
2 # dove s e 'match', o la str(P) dove P e una produzione di G e tid e
  una TopDownInstantaneousDescription
3 # ottenuta applicando il match o la predizione secondo P a curr
4
5 def next_instdescribers(instdescriber):
6     if instdescriber.is_done(): return []
7     G = instdescriber.G
8     top = instdescriber.top()
9     if top in G.T:
10         return [('match', instdescriber.match())] if top == instdescriber.head()
11         else []
12     else:
13         return [(str(P), instdescriber.predict(P)) for P in
14                 filter(Production.such_that(lhs = top), G.P)]

```

Come detto la prima cosa che vogliamo fare è una derivazione in ampiezza, metto in una coda i nodi che ho visitato sotto l'assunzione che ho visto il nodo che ha la stessa configurazione di pila e nastro. Il problema se abbiamo ricorsioni è che potremmo andare all'infinito perchè il grafo diventerebbe infinito, quindi ci fermiamo al primo stato istantaneo buono (il verde) questo e' quello che fa il parametro `first_only`.

```

1 def breadth_first(G, word, verbose = False, first_only = True):
2     graph = prepare_graph()
3     q = Queue()
4     q.enqueue(TopDownInstantaneousDescription(G, word))
5     seen = {q.Q[0]}
6     derivations = []
7     while q:
8         if first_only and derivations: break
9         if verbose:
10             for i in q: print(i)
11             print('-', * 60)
12         curr = q.dequeue()
13         for what, nxt in next_instdescribers(curr):
14             graph.edge(curr, nxt, gv_args={'label': what})
15             if nxt.is_done(): derivations.append(nxt.steps)
16             if not nxt in seen:
17                 seen.add(nxt)
18                 q.enqueue(nxt)
19     return derivations, graph

```

Ora usiamo una visita in profondità, modifichiamo il tappo di visita, non mi fermiamo alla prima ma contiamo i passi che faccio, se li supero mi fermo.

```

1 def depth_first(G, word, verbose = False, max_steps = -1, use_seen =
  True):
2     graph = prepare_graph()
3     s = Stack()
4     s.push(TopDownInstantaneousDescription(G, word))
5     seen = {s.S[0]}
6     derivations = []
7     steps = 0
8     while s:
9         if steps > max_steps > -1: break
10        steps += 1
11        if verbose:
12            for i in s: print(i)
13            print('-', * 60)
14        curr = s.pop()
15        for what, nxt in next_instdescriptors(curr):
16            graph.edge(curr, nxt, gv_args={'label': what})
17            if nxt.is_done(): derivations.append(nxt.steps)
18            if not nxt in seen:
19                if use_seen: seen.add(nxt)
20                s.push(nxt)
21    return derivations, graph

```

Con questo nuovo algoritmo notiamo che la coda non contiene più tanti elementi, ma 2/3 alla volta. Vediamo che il grafo scende molto più di prima.

Il problema è proprio la ricorsione, cosa succede se ho regole in cui ho S come ricorsione a sinistra? del tipo:

```

1 S -> S a | b

```

Questo produce una situazione in cui, anche se ho una parola breve, facendo una visita in ampiezza la parola la trovo, ma se faccio una visita in profondità le derivazioni verdi non le trova, perchè non ha un loop nella pila (trova i rossi quando ritrova stati uguali) la pila aumenta all'infinito e non trova mai la parola.

Un modo per correggere il tiro c'è, in particolare la presenza di epsilon regole è drammatica (la pila aumenta senza consumare il nastro), però se evitiamo le epsilon regole possiamo, data una grammatica monotonica (in cui la forma sentenziale non si può ridurre, per noi è la pila) possiamo fermarci quando la pila è troppo lunga, ad esempio se devo generare pippo e sulla pila ho più di 5 non terminali non posso generare pippo, ma genererò una parola di più di 5 caratteri. Quindi nella visita se mi infilo in una circostanza in cui so che non incontrerò mai un nodo verde allora posso fermarmi. Allora posso considerare una produzione produttiva se la sua produzione è più corta di quello che resta da mangiare.

```

1 def next_instdescriptors(curr):
2     def productive(pair):
3         _, instdescr = pair
4         return len(instdescr.stack) <= (len(instdescr.tape) -
          instdescr.head_pos)
5     return list(filter(productive, original_next_instdescriptors(curr)))

```

Considero produttive solo le situazioni in cui sulla pila ho un numero di terminali sensato per la parola che ho da generare. Quindi così anche con una visita in profondità

riesco a trovare la parola. Notiamo che con la perdita delle epsilon regole perdiamo la possibilità di avere una produzione infinita.

Una cosa carina è che potremmo cambiare la scelta di dove andare solamente basandoci sul nostro passato (se sono arrivato allo stesso punto), posso vedere di togliere questa condizione, ovvero non mi pongo il problema di arrivare in un posto nello stesso modo (non mi interessa il passato). Ovvero se introduciamo l'ambiguità quello che troveremo e' alla fine sempre tutte le derivazioni con la differenza che potremmo avere archi doppi allo stesso nodo, questo perche' potrei arrivare a quel nodo ma con storie diverse quindi togliendo la condizione di uguaglianza tra istantanee potrei avere piu' archi che arrivano allo stesso nodo.

3.4 Parsing Ricorsivo Discendente

3.4.1 Eliminare Ricorsione a Sinistra

Eliminare la ricorsione è una cosa molto comoda anche per poter usare alcuni parser generator che funzionano solo se non c'è la ricorsione quindi è un trucco su cui vale la pena fare un discorso.

Il caso più elementare, se abbiamo un terminale che ha una ricorsione a sinistra questa cosa vorrebbe generare un ciclo, una lista finita di elementi:

$$A \rightarrow A\alpha_1 | A\alpha_2 \dots | \beta_1 | \beta_2 \dots$$

Quindi quello che faremo è introdurre dei nuovi terminali, la testa si occupa di generare i beta, poi similmente si definisce un A di coda che permette di ottenere gli alpha:

$$A_H \rightarrow \beta_1 | \beta_2 \dots B_T \rightarrow \alpha_1 | \alpha_2 \dots$$

Poi in realtà per generare la lista devo riportare la ricorsione, la spostiamo a destra inserendo un nuovo terminale:

$$A_{TS} \rightarrow A_T A_{TS}$$

Dopo di che posso generare uno dei simboli A:

$$A \rightarrow A_H A_{TS} | A_H$$

Abbiamo risolto la ricorsione diretta ma manca quella indiretta, una cosa del genere:

$$A \rightarrow B\alpha B \rightarrow C\gamma C \rightarrow A\beta$$

L'idea in questo caso è ancora strutturalmente semplice. Un modo semplice per risolvere il problema sarebbe quello di ordinare i non terminali, se potessi chiamarli:

$$A_1 < A_2 < \dots < A_n$$

Il problema di questo caso è che parto da un non terminale e torno allo stesso. Quello che voglio riscrivere tutte le produzioni per cui quelle in cui non c'è un non terminale come primo simbolo deve essere minore di quello che c'è a destra (voglio evitare catene):

$$A_i \rightarrow A_j \alpha_i < j$$

Il procedimento è iterativo e si parte con $i = 1$, in questo caso non voglio avere cose della ricorsione diretta $A_1 \rightarrow A_1\alpha_1|\alpha_2\dots$ quindi lo abbiamo fatto prima possiamo quindi creare una grammatica senza questa ricorsione. Ora passo a $i = 2$ e qui avrò cose del genere:

$$A_2 \rightarrow A_1\alpha A_2\alpha A_3\alpha\dots$$

Prendo questa grammatica e sostituisco gli A_1 con quelli che abbiamo sistemato prima. Andando avanti così riusciamo ad avere una grammatica G senza ricorsione a sinistra.

3.4.2 Prefix-Free

Adesso c'è un'ulteriore richiesta che è più sottile, vogliamo parlare dei prefix-free, non è possibile avere un prefisso proprio dentro un linguaggio. Possiamo dire che una grammatica è prefix free se e solo se tutti i non terminali della grammatica sono prefix free, ossia A non deriva mai sia una parola che un suo prefisso. Non è detto che una grammatica si possa trasformare in prefix free, quindi noi lavoreremo sull'ipotesi che la grammatica sia prefix free.

3.4.3 Parsing Ricorsivo Discendente

L'idea è che vorremmo scrivere una funzione parse che prende in input un simbolo x e un pezzo di forma sentenziale w e restituisce una coppia booleano w' tale per cui se per come è fatta la grammatica x effettua il parsing di un prefisso di w allora restituisce true e il pezzo non parsato di w (nel caso in cui x sia un terminale significa che x deve essere la prima lettera di w).

Vogliamo provare a scrivere il parsing per le espressioni più e meno:

```
1  E -> E + T | E - T | T
2  T -> t
```

Partendo da una grammatica ricorsiva così si può trasformare in una grammatica non ricorsiva:

```
1  """
2  E -> Eh Ets | Eh
3  Eh -> T
4  Ets -> Et Ets | Et
5  Et -> + T | - T
6  T -> t
7  """
```

Questa grammatica non è prefix free ma noi vogliamo renderla prefix free, in questo caso possiamo mettere un tappo, sostanzialmente vogliamo vedere se dopo Eh mi devo fermare o no, mi fermo se trovo un cancelletto:

```
1  G = Grammar.from_string("""
2  E -> Eh Ets | Eh #
3  Eh -> T
4  Ets -> Et Ets | Et #
5  Et -> + T | - T
6  T -> t
7  """)
```

Possiamo ragionare per casi, trattando a parte il caso dei non terminali e poi ogni terminale a parte (una sorta di switch). I non terminali devo considerarli sulla scorta delle sue alternative (quel non terminale può produrre questo o quello), nel caso di T l'unica cosa che posso fare per dirti se funziona è vedere se t funziona (richiamando il parse)

```

1 @show_calls(True)
2 def parse(X, rest):
3
4     if X in G.T:
5
6         if rest and rest[0] == X: return True, rest[1:]
7     elif X == 'T': # T -> t
8
9         return parse('t', rest)
10
11    elif X == 'Eh': # Eh -> T
12
13        return parse('T', rest)
14
15    elif X == 'Et': # Et -> + T | - T
16
17        s, r = parse('+', rest)
18        if s: return parse('T', r)
19        s, r = parse('-', rest)
20        if s: return parse('T', r)
21
22    elif X == 'Ets': # Ets -> Et Ets | Et #
23
24        s0, r0 = parse('Et', rest)
25        if s0:
26            s1, r1 = parse('#', r0)
27            if s1: return True, r1
28            return parse('Ets', r0)
29
30    elif X == 'E': # E -> Eh Ets | Eh #
31
32        s0, r0 = parse('Eh', rest)
33        if s0:
34            s1, r1 = parse('#', r0)
35            if s1: return True, r1
36            return parse('Ets', r0)
37
38    return False, rest

```

Bello però noi vogliamo l'albero di parsing, modifichiamo la funzione affinché ritorni anche i passi left-most con cui x produce il prefisso (lo produce usando regola n):

```

1 # ora parse restituisce: succ, rest, steps
2
3 @show_calls(True)
4 def parse(X, rest):
5
6     if X in G.T:
7
8         if rest and rest[0] == X: return True, rest[1:], None

```

```

9
10 elif X == 'T':
11
12     s, r, _ = parse('t', rest)
13     if s: return True, r, [7]
14
15 elif X == 'Eh':
16
17     s, r, p = parse('T', rest)
18     if s: return True, r, [2] + p
19
20 elif X == 'Et':
21
22     s, r, _ = parse('+', rest)
23     if s:
24         s, r, p = parse('T', r)
25         if s: return True, r, [5] + p
26     s, r, _ = parse('-', rest)
27     if s:
28         s, r, p = parse('T', r)
29         if s: return True, r, [6] + p
30
31 elif X == 'Ets':
32
33     s0, r0, p0 = parse('Et', rest)
34     if s0:
35         s1, r1, _ = parse('#', r0)
36         if s1: return True, r1, [4] + p0
37         s2, r2, p2 = parse('Ets', r0)
38         if s2: return True, r2, [3] + p0 + p2
39
40 elif X == 'E':
41
42     s0, r0, p0 = parse('Eh', rest)
43     if s0:
44         s1, r1, _ = parse('#', r0)
45         if s1: return True, r1, [1] + p0
46         s2, r2, p2 = parse('Ets', r0)
47         if s2: return True, r2, [0] + p0 + p2
48
49 return False, rest, []

```

3.4.4 Generazione automatica del parser

Non è così impensabile partire da una grammatica G e fare un programma che produce la funzione `parse`, perchè lo abbiamo fatto applicando solamente un pattern (gli if della soluzione sono prodotti, uno per ciascun simbolo, considerando in sequenza le possibili alternative (i lati destri delle produzioni) e restituendo successo al primo tentativo valido), quindi quello che vogliamo fare è trovare il modo di creare una funzione `parse` che funziona per una grammatica. Questa cosa si chiama *parser generator*.

Un esempio e' il simbolo `Ets` le cui due alternative sono `Et Ets` e `Et # ...` supponiamo di scaricarne la soluzione a due funzioni denominate `Ets_alt0` e `Ets_alt1`:

```

1 def parse(X, rest):
2
3     # ...
4
5     if X == 'Ets':
6         succ_alt, rest_alt = Ets_alt0(rest)
7         if succ_alt: return True, rest_alt
8         succ_alt, rest_alt = Ets_alt1(rest)
9         if succ_alt: return True, rest_alt
10        return False, rest
11
12    # ...
13
14 def Ets_alt0(rest):
15     succ, rest = parse('Et', rest)
16     if not succ: return False, rest
17     succ, rest = parse('Ets', rest)
18     if not succ: return False, rest
19     return True, rest
20
21 def Ets_alt1(rest):
22     succ, rest = parse('Et', rest)
23     if not succ: return False, rest
24     succ, rest = parse('#', rest)
25     if not succ: return False, rest
26     return True, rest

```

Quello che facciamo in generale è che ogni volta che un non terminale aveva delle alternative le abbiamo eseguite in cascata invocando una certa funzione che usava quella alternativa, se restituiva true tornavamo altrimenti andavamo nell'else. Ci servono due funziononi di utilità: la prima per indentare un blocco di testo al livello di indentazione specificato e la seconda per tradurre il codice sorgente di una funzione Python di nome parse in codice eseguibile:

```

1 def indent_at(level, source):
2     return indent(dedent(source), ' ' * level).strip('\n')
3
4 def make_parse(source):
5     glb = {'show_calls': show_calls}
6     exec(source, glb)
7     return glb['parse']

```

Siamo pronti per procedere, useremo le f-stringhe per maggior compattezza:

```

1 def make_parse_source(G):
2
3     # prima le definizioni delle funzioni A_altN
4
5     defs = []
6     for A in G.N:
7         for n, a in enumerate(G.alternatives(A)):
8             # la funzione "{A}_alt{n}"
9             defs.append(f'def {A}_alt{n}(rest):')
10            for X in a:
11                # una coppia di linee per ogni simbolo
12                defs.append(indent_at(1, f'''

```

```

13     succ, rest = parse('{X}', rest)
14     if not succ: return False, rest
15     ''')
16     defs.append(indent_at(1, 'return True, rest'))
17
18 # poi gli if, uno per terminale
19
20 ifs = []
21 for A in G.N:
22     # l'if "X == '{A}'"
23     ifs.append(f"if X == '{A}':")
24     for n, _ in enumerate(G.alternatives(A)):
25         # una coppia di linee, una per alternativa
26         ifs.append(indent_at(1, f'''
27             succ_alt, rest_alt = {A}_alt{n}(rest)
28             if succ_alt: return True, rest_alt
29             '''))
30     ifs.append(indent_at(1, 'return False, rest'))
31
32 # in fine i terminali
33
34 ifs.append(indent_at(0, f'''
35     if X in {G.T}:
36         if rest and rest[0] == X: return True, rest[1:]
37         return False, rest
38     '''))
39
40 # e ora mettiamo tutto assieme
41
42 parse = '\n'.join((
43     indent_at(0, '''
44         @show_calls(True)
45         def parse(X, rest):
46             '''),
47     indent_at(1, '\n'.join(defs)),
48     indent_at(1, '\n'.join(ifs))
49 ))
50
51 return parse

```

Faremo per tutti i non terminali genereremo una serie di or e poi una funzione specifica che fa la concatenazione. L'unica cosa che ci serve è mettere a posto l'indentazione. Python ci permette, data una stringa sorgente renderla eseguibile, prende un sorgente di una parse e restituisce una funzione che fa il parsing (reflection di python). Se generiamo e stampiamo il sorgente otteniamo questo:

```

1 source = make_parse_source(G) # G e la grammatica prefix-free
2     dell'inizio
3
4 print(source)
5
6 @show_calls(True)
7 def parse(X, rest):
8     def Ets_alt0(rest):
9         succ, rest = parse('Et', rest)
10        if not succ: return False, rest
11        succ, rest = parse('Ets', rest)

```

```

10     if not succ: return False, rest
11     return True, rest
12 def Ets_alt1(rest):
13     succ, rest = parse('Et', rest)
14     if not succ: return False, rest
15     succ, rest = parse('#', rest)
16     if not succ: return False, rest
17     return True, rest
18 def E_alt0(rest):
19     succ, rest = parse('Eh', rest)
20     if not succ: return False, rest
21     succ, rest = parse('Ets', rest)
22     if not succ: return False, rest
23     return True, rest
24 def E_alt1(rest):
25     succ, rest = parse('Eh', rest)
26     if not succ: return False, rest
27     succ, rest = parse('#', rest)
28     if not succ: return False, rest
29     return True, rest
30 def Et_alt0(rest):
31     succ, rest = parse('+', rest)
32     if not succ: return False, rest
33     succ, rest = parse('T', rest)
34     if not succ: return False, rest
35     return True, rest
36 def Et_alt1(rest):
37     succ, rest = parse('-', rest)
38     if not succ: return False, rest
39     succ, rest = parse('T', rest)
40     if not succ: return False, rest
41     return True, rest
42 def Eh_alt0(rest):
43     succ, rest = parse('T', rest)
44     if not succ: return False, rest
45     return True, rest
46 def T_alt0(rest):
47     succ, rest = parse('t', rest)
48     if not succ: return False, rest
49     return True, rest
50 if X == 'Ets':
51     succ_alt, rest_alt = Ets_alt0(rest)
52     if succ_alt: return True, rest_alt
53     succ_alt, rest_alt = Ets_alt1(rest)
54     if succ_alt: return True, rest_alt
55     return False, rest
56 if X == 'E':
57     succ_alt, rest_alt = E_alt0(rest)
58     if succ_alt: return True, rest_alt
59     succ_alt, rest_alt = E_alt1(rest)
60     if succ_alt: return True, rest_alt
61     return False, rest
62 if X == 'Et':
63     succ_alt, rest_alt = Et_alt0(rest)
64     if succ_alt: return True, rest_alt
65     succ_alt, rest_alt = Et_alt1(rest)

```

```

66     if succ_alt: return True, rest_alt
67     return False, rest
68 if X == 'Eh':
69     succ_alt, rest_alt = Eh_alt0(rest)
70     if succ_alt: return True, rest_alt
71     return False, rest
72 if X == 'T':
73     succ_alt, rest_alt = T_alt0(rest)
74     if succ_alt: return True, rest_alt
75     return False, rest
76 if X in frozenset({'+', '-', 't', '#'}):
77     if rest and rest[0] == X: return True, rest[1:]
78     return False, rest

```

Notiamo che è una ricorsione con backtracking, non scendo all'infinito ma mi fermo prima in alcuni casi.

La cosa drammatica di questo parser è che se ho due alternative tengo sempre buona la prima, ma se il linguaggio non è context free potrei entrare dentro il prefisso e poi dentro lì andrei avanti all'infinito. In questo esempio vediamo che parsando alla fine ottengo true con una lista non vuota, quindi sembrerebbe che non ha parsato:

```

1  # una grammatica non prefix-free (L = {a, ab})
2
3  G = Grammar.from_string("""
4  S -> a | a b
5  """)
6
7  # costruiamo il parser e tentiamo il parse
8
9  source = make_parse_source(G)
10 parse = make_parse(source)
11 parse('S', 'ab')
12
13 (True, 'b')

```

Il problema che non è questa la strada, di fronte ad una disgiunzione $A \rightarrow B|C$ non posso prendere per buona solo la produzione B ed ignorare il fatto che potrebbe produrre anche C (che potrebbe essere meglio), ed è ovviamente un grande limite. Quello che si potrebbe fare è avere un parsing con le continuazioni, sostanzialmente restituisco nel parse anche una funzione che prova tutte le alternative. Il problema di questa cosa è che si risolve il problema ma non è più lineare, perchè fa tutte le alternative.

In buona sostanza avere un parsing ricorsivo discendente è fighissimo perchè è lineare, semplice da implementare e permette di avere già in mano l'interprete. Ad esempio il parsing ricorsivo discendente è quello di Python. Forse se stiamo costruendo il nostro DSL, con delle regole semplici (comandare drone, irrigazione ...), possiamo permetterci di avere un parsing ricorsivo discendente, ma se vogliamo fare un parser generico per un linguaggio generico non possiamo permetterci di avere un parsing ricorsivo discendente.

3.5 Parsing Bottom-Up

La realizzazione di parsing bottom up veniva più facile negli anni 70 infatti si è partiti da questo tipo di parser, ragionare su parsing concreti per le grammatiche context-free

deterministiche era più facile. Aggiustare una grammatica per farlo funzionare in questo contesto è più difficile questo perchè funziona al contrario rispetto agli altri parser. Interessante perchè è a livello storico è il primo parsing che si occupa di un grosso numero di grammatiche in tempo lineare ma è doloroso perchè richiede uno sforzo al progettista di grammatiche.

Abbiamo sempre in mente la pila (che contiene i simboli), il nastro e il controllo. L'idea è mettere sopra il nastro degli alberi, costruire degli alberi che rappresentano delle porzioni della parola (le derivazioni) e li uniscono verso l'alto per avere l'albero di derivazione costruito dal fondo all'alto. Quindi il punto cruciale di questo algoritmo è cominciare a guardare la base e poi la foresta che mi sono costruito sul nastro, quindi sulla pila devo tenere oltre ai terminali e non terminali devo tenere anche questi alberi. Allora la cosa più comoda da considerare è che di fatto nella pila voglio tenere le cime degli alberi della foresta, ma correlata a questa informazione i passi che ho fatto.

3.5.1 Shift e Reduce

Quando vedo un carattere sul nastro posso:

- Metterlo sulla cima della pila, facendo uno shift
- Riconoscere che sulla pila c'è il contenuto di una regola produttiva, nella grammatica c'era da qualche parte $A \rightarrow CD$ allora potrei togliere dalla pila C e D e mettere A, quindi ho fatto una riduzione (reduce). Da notare che in questo esempio eravamo in una situazione in cui sulla pila c'erano D e C che a loro volta rappresentavano un albero di derivazione.

Quindi il controllo cerca regole della grammatica il cui lato destro è l'unione di elementi sulla pila. Questa configurazione non è proprio un'automa a pila, perchè non guardo solo la cima della pila, ma spio tutta la pila. Il che sembrerebbe suggerire che questa è una macchina di Turing, ma scopriremo che non è così. Quello che va colto è che è un approccio inerentemente più complesso perchè l'indecisione è più grande, perchè non solo devo decidere in caso di regole multiple cosa fare ma devo anche decidere se fare uno shift o una riduzione e anche nel caso della riduzione devo decidere quale regola applicare.

Adesso come sempre se abbiamo in testa una cosa che non è deterministica non riusciamo nella pratica a trovare la soluzione, mi immagino di avere una forma descrittiva dello stato della computazione (una descrizione che racconta i passi della computazione), produrre una funzione next che dirà data una delle circostanze cosa fare (uno shift e quale reduce fare) che mi dirà quali sono le successive istantanee oppure che nessun lato destro della grammatica fa parte della pila (siamo rovinati) o che abbiamo finito il nastro (abbiamo finito di parsare).

A questo punto simulo, parto da uno stadio iniziale e poi con una modalità breadth-first (allargo la vista) o con modalità depth-first (più pericolosa perchè potrei andare giù all'infinito).

Vediamo una grammatica e facciamo un atto di fede in cui la pila è unbound (perchè ci teniamo dentro anche i passi di derivazione):

```
1 G = Grammar.from_string(""  
2 S -> A C  
3 A -> a b
```

```

4 C -> c
5 """)
6
7 i = BottomUpInstantaneousDescription(G, 'abc')
8 # (), , |abc
9
10 i = i.shift()
11 display(i, side_by_side(i.stack))
12 # (), (a), a|bc
13
14 i = i.shift()
15 display(i, side_by_side(i.stack))
16 # (), (a)(b), ab|c
17
18 i = i.reduce(G.P[1])
19 display(i, side_by_side(i.stack))
20 # (A -> a b,), (A: (a), (b)), ab|c
21
22 . . .
23
24 i = i.reduce(G.P[0])
25 display(i, side_by_side(i.stack))
26 # (S -> A C, C -> c, A -> a b), (S: (A: (a), (b)), (C: (c))), abc|

```

Vediamo che la descrizione istantanea è come l'altra, ho i passi di derivazione, la pila vuota, la testina e il nastro. A botte di shift e reduce portiamo un po' di simboli sulla pila e quando vedo che sulla pila ci sono delle regole destre della grammatica tolgo quelle teste e ci metto l'albero di derivazione.

Adesso ragioniamo su come fare la funzione next, l'aspetto cruciale è che abbiamo un automa che ha delle primitive che si riflettono sulla sua natura di automa a pila, dopo di che siccome per ogni condizione in cui si trova ho un certo insieme finito di possibilità (non ristretto ad 1) posso fare una simulazione che devo fare con la funzione next. Il passo di riduzione è più difficile da implementare perchè devo prendere tutto lo stack, corrisponde ad una lettura completa della pila (dal punto di vista di programmazione ad oggetti svelo tutto lo stato dell'oggetto). All'occhio che qui non ci sono le epsilon regole.

```

1 G = Grammar.from_string('S -> a S b | S a b | a a a')
2 derivations, graph = breadth_first(G, list('aaaab'), True, False) #
   raccolgo tutte le derivazioni

```

```

1 # restituisce un elenco di coppie (s, bid)
2 # dove s e' 'shift', o la str(P) dove P e' una produzione di G e bid e'
   una BottomUpInstantaneousDescription
3 # ottenuta applicando lo shift o la riduzione usando P a curr
4
5 def next_instdescrs(curr):
6
7     if curr.is_done(): return []
8
9     instdescrs = []
10
11     # shift
12     if curr.head_pos < len(curr.tape): instdescrs.append(('shift',
        curr.shift()))

```

```

13     if curr.G.has_epsilon_productions(): instdescriptors.append(('e-shift',
14                                     curr.shift(False)))
15
16     # reduce
17     tops = tuple(t.root for t in curr.stack)
18     for P in filter(Production.such_that(rhs_is_suffix_of = tops),
19                     curr.G.P):
20         instdescriptors.append((str(P), curr.reduce(P)))
21
22     return instdescriptors
23
24 # una funzione che associa ad ogni descrizione un colore
25 # a seconda che sia accettante (verde), o non abbia stati prossimi
26 # (rosso)
27 def node2color(tid):
28     return {'color': 'green' if tid.is_done() else 'black' if
29             next_instdescriptors(tid) else 'red'}

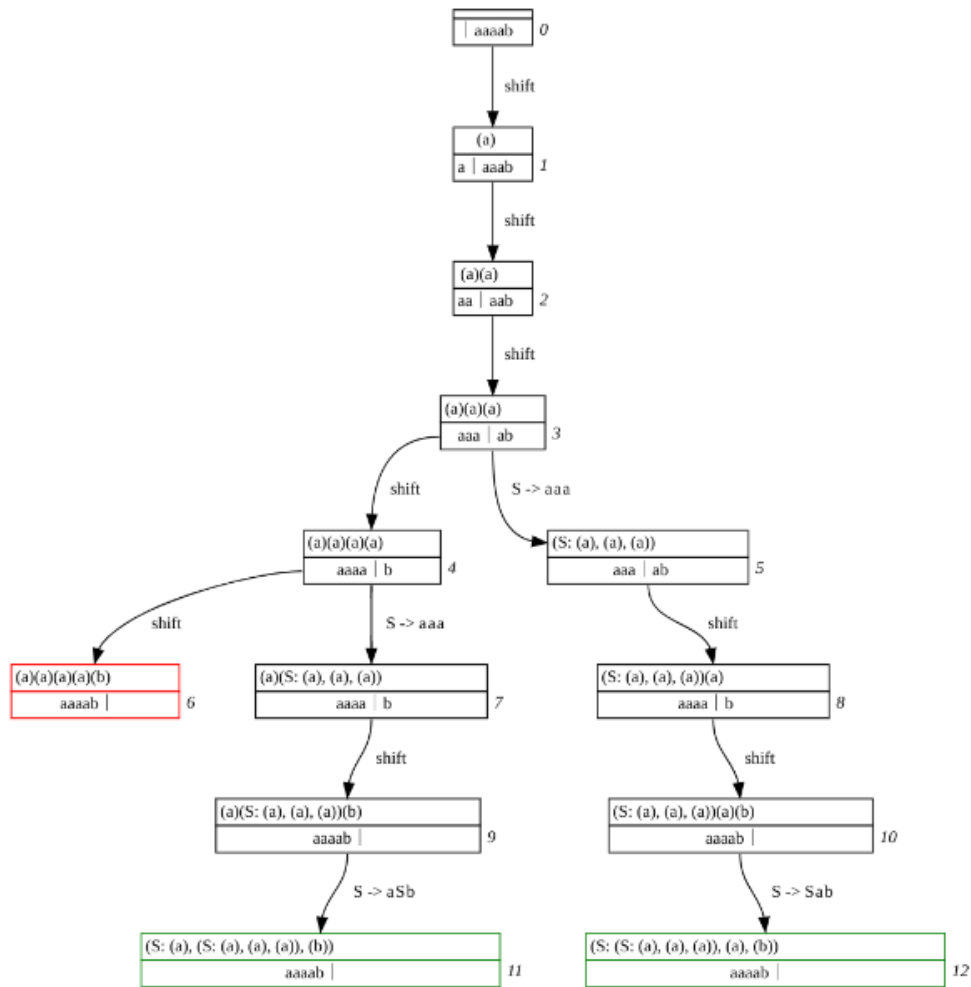
```

Possiamo usare lo stesso codice dell'altra volta per fare la visita in ampiezza, abbiamo visto un esempio data una grammatica con ricorsione a sinistra che produce due derivazioni right-most diverse. Vediamo che come l'altra volta abbiamo il parametro `first_only` che ci permette di fermarci alla prima derivazione verde.

```

1 def breadth_first(G, word, verbose = False, first_only = True):
2     graph = ComputationGraph(node2color)
3     q = Queue()
4     q.enqueue(BottomUpInstantaneousDescription(G, word))
5     derivations = []
6     while q:
7         if first_only and derivations: break
8         if verbose:
9             for i in q: print(i)
10            print('-', * 60)
11        curr = q.dequeue()
12        for what, nxt in next_instdescriptors(curr):
13            if nxt.is_done(): derivations.append(nxt.steps)
14            if not graph.seen(nxt):
15                q.enqueue(nxt)
16            graph.step(curr, nxt, what)
17    return derivations, graph

```



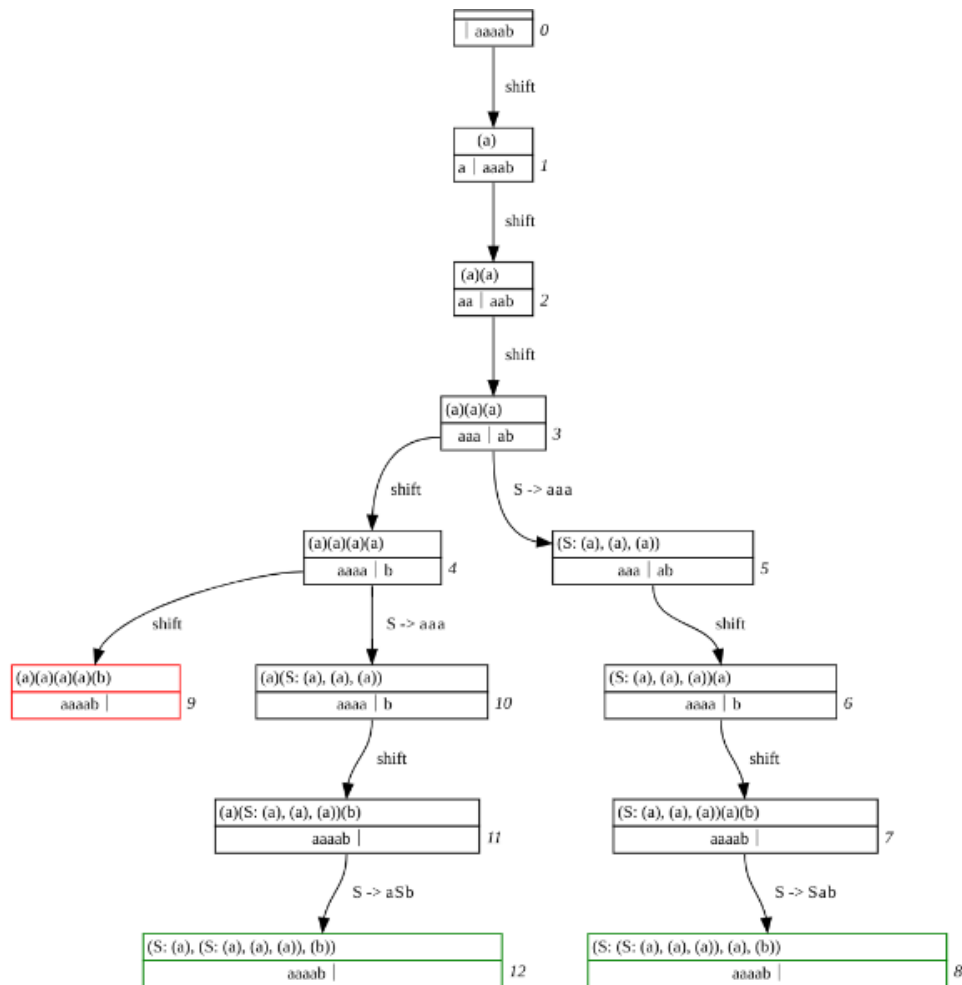
Anche l'approccio con visita in profondità è lo stesso dell'altra volta:

```

1 def depth_first(G, word, verbose = False, max_steps = -1, use_seen =
  True):
2     graph = ComputationGraph(node2color)
3     s = Stack()
4     s.push(BottomUpInstantaneousDescription(G, word))
5     derivations = []
6     steps = 0
7     while s:
8         if steps > max_steps > -1: break
9         steps += 1
10        if verbose:
11            for i in s: print(i)
12            print('-', * 60)
13        curr = s.pop()
14        for what, nxt in next_instdescr(curr):
15            if nxt.is_done(): derivations.append(nxt.steps)
16            if not (use_seen and graph.seen(nxt)):
17                s.push(nxt)
18            graph.step(curr, nxt, what)
19    return derivations, graph

```

Vediamo un esempio che anche con una grammatica apparentemente elementare ed



una stringa corta ha dei problemi non visibili ad occhio nudo, se la proviamo con la visita in profondità è vero che arriva alla soluzione ma sbaglia un sacco di strade.

```

1 # fig. 7.8, pag. 204
2
3 G = Grammar.from_string("""
4 S -> E
5 E -> E Q F | F
6 F -> a
7 Q -> + | -
8 """)

```



Questo algoritmo come abbiamo visto è più difficile da ragionarsi sopra in particolare perchè restano due brutte cose da sistemare:

- Devo leggere tutta la pila
- Devo tenere tutti gli alberi

Ma anche supponendo che questi problemi teorici vengano risolti resta il fatto che purtroppo in questo caso le grammatiche che producono il disastro sono molto più difficili da vedere rispetto alle grammatiche che producono il disastro per i parser top down. Esistono strumenti automatici che ci dicono se la grammatica è ambigua o meno per i parser bottom up (LR(0)) ma è comunque doloroso modificare quelle grammatiche malate. Notiamo che TopDown e BottoUp si usano per grammatiche deterministiche e che senza ambiguità hanno costo computazionale $O(n)$, quindi non è un problema di efficienza ma di correttezza.

Dal lato dei parser Bottom Up ci sarebbe l'algoritmo CYK (che usiamo per le grammatiche generali ed ha costo computazionale n^3) bello, che è l'algoritmo di Early che funziona per le grammatiche generali che in caso di non ambiguità per le grammatiche deterministiche (quelle che usiamo TopDown e BottoUP) diventa $O(n)$.

3.6 Parsing top-down direzionale deterministico

Vogliamo metterci nella condizione per cui la cosa che adoperiamo che ha la testina la pila e la parola ad un certo punto vede la cima della pila e la prossima lettera della parola (sotto la testina) il controllo può fare la predizione. La necessità di seguire tutti i possibili cammini nel grafo che simula la computazione del NPDA deriva dal fatto che per un certo simbolo A in cima alla pila e il primo terminale b sotto la testina, ci sono più produzioni da considerare. In buona sostanza vogliamo una tabella, che dipende dalla grammatica, che mi dice la corrispondenza tra simboli terminali e non terminali e grazie a questa corrispondenza riesco a derivare subito la produzione. Grazie a questa tabella implementare il controllo dell'automa è una banalità, se la cima è un terminale ti dico se c'è un match o no, se la cima è un non terminale ti dico se posso fare una predizione o meno. La magia grande è la tabella che deve funzionare, non deve contenere nessuna produzione se sono in un posto in cui non dovrei essere.

Adesso il punto è come produciamo questa tabella, in realtà la abbiamo già vista, perchè ad un certo punto abbiamo giocato con una grammatica che era così:

```

1  S -> aB
2  B -> b | aBb

```

Funziona bene perchè il primo carattere di ogni produzione è diverso, se sono in una circostanza per cui per ogni possibilità mi porta ad una produzione con il primo carattere diverso allora posso costruire la tabella:

	S	B
a	S -> ab	B -> b
b		B -> aBb

Queste grammatiche si chiamano SSL(1), dove 1 sta per il numero di simboli che guardiamo dopo la testina.

Per costruire la tabella prendo le produzioni della grammatica, le smonto come lato sinistro e letterina e suffisso e la produco così (se c'è un tentativo di riassegnamento mi avvisa che non è una grammatica che posso trattare), per ogni produzione $A \rightarrow b\delta$ basta considerare il primo simbolo che è terminale e distingue le varie alternative:

```

1  def compute_simple_table(G):
2
3  TABLE = Table(2, no_reassign = True) # no_reassign sara chiarito in
    seguito
4
5  for P in G.P:
6      A, (b, +d) = P
7      TABLE[A, b] = P
8
9  return TABLE

```

Ma se la grammatica non è semplice? nella costruzione della tabella mi avverte che c'è un conflitto (cerca di mettere due entry per la stessa coppia).

Questo modo di costruire i parser è facile ma utile, ad esempio è la notazione prefissa (notazione polacca) che è una notazione che non ha bisogno di parentesi, perchè il parser è in grado di capire la precedenza degli operatori. Ad esempio:

```

1  G = Grammar.from_string("""
2  E -> + E E | - E E | * E E | / E E | t
3  """)

```

Questo modo di scrivere le espressioni è molto comodo perchè è immediato parsare le associazioni e le precedenze, ad esempio:

$$\begin{aligned}
 * + 3 2 4 &\rightarrow (3 + 2) * 4 \\
 + 3 * 2 4 &\rightarrow 3 + (2 * 4)
 \end{aligned}$$

Questa costrizione è molto stringente, non avere due simboli uguali in due produzioni diverse è una cosa che non è sempre possibile, ad esempio vorrebbero dire non poter avere if e if else. Allora una cosa su cui possiamo ragionare è questa, supponiamo di sapere per ogni forma sentenziale qual'è la prima lettera che compare nella derivazione. Mi interessa conoscere data una forma sentenziale i prefissi, di tutte le parole che questa forma sentenziale può produrre quali sono i primi caratteri di queste derivazioni. Se avessi questa roba qui potrei costruire la tabella in un'altra ottica, metto in tabella le produzioni basandomi sui first, ad esempio se voglio collocare $A \rightarrow w$ e so che $\text{first}(w) = a, b, f$ allora la tabella sarebbe così:

	A
a	$A \rightarrow w$
b	$A \rightarrow w$
...	...
f	$A \rightarrow w$

Il first di una forma sentenziale nella forma in cui nessuna delle parti può scomparire (in quanto non ci sono epsilon regole) è il primo simbolo che compare nella derivazione. Nel riempire la tabella posso fare una fattorizzazione, prendo la A e la spacco e poi prendo tutte le lettere che vengono prodotte da B e riempio la tabella:

```

1  def compute_table(G):
2
3  TABLE = Table(2, no_reassign = True)

```

```

4
5 FIRST = compute_first(G)
6
7 for P in G.P:
8     A, (B, *d) = P
9     for a in FIRST[B]:
10         TABLE[A, a] = P
11
12 return TABLE

```

Con questa cosa del first riesco a vedere i conflitti un po' in avanti, un conflitto lo ho quando due produzioni hanno lo stesso first.

Si suggerirebbe un approccio ricorsivo, questo per guardare i first, ricordando che questo è perchè non abbiamo le epsilon regole. Il calcolo di $FIRST(\beta)$ dove si ponda $\beta = C\delta$, può essere semplificato osservando che $FIRST(C\delta) = FIRST(C)$.

```

1 def compute_first(G):
2
3     def recursive_first(B): # B e G.T | G.N
4         return union_of(
5             recursive_first(C)
6             for C, *d in G.alternatives(B) if B != C # occhio alla ricorsione
7             a sinistra!
8             ) if B in G.N else {B}
9
10    FIRST = Table(1)
11    for X in G.N | G.T: FIRST[X] = recursive_first(X)
12
13    return FIRST

```

Vediamo un esempio con una grammatica di un ipotetico sistema esperto nel quale prima si indica un fatto, una sequenza di fatti è una sessione e poi si termina con una domanda:

```

1 # fig. 8.7, pag. 240
2
3 G = Grammar.from_string("""
4 Session -> Fact Session | Question | ( Session ) Session
5 Fact -> ! STRING
6 Question -> ? STRING
7 """)

```

Ricorsivamente possiamo costruire i first, l'idea è che posso usare una regola che ha sessione a sinistra solo se sul nastro vedo uno di questi ?, (, !. E a questo punto posso costruire la tabella di derivazione, in questo caso non ci sono conflitti e quindi posso costruire la tabella:

Question	?
Fact	!
Session	(, !, ?

L'uso della tabella first mi consente di sapere, non tanto sulla scorta della prima letterina dei lati destri, ma sulla scorta dei first della grammatica (le letterine), se questa

tabella non ha conflitti sono a posto, posso chiamare questa roba LL(1) e posso chiamare il parser che funziona:

1. first
2. table if no conflitti:
3. parser

Abbiamo trovato un parser deterministico lineare, lineare perchè legge ogni lettera una volta sola.

```

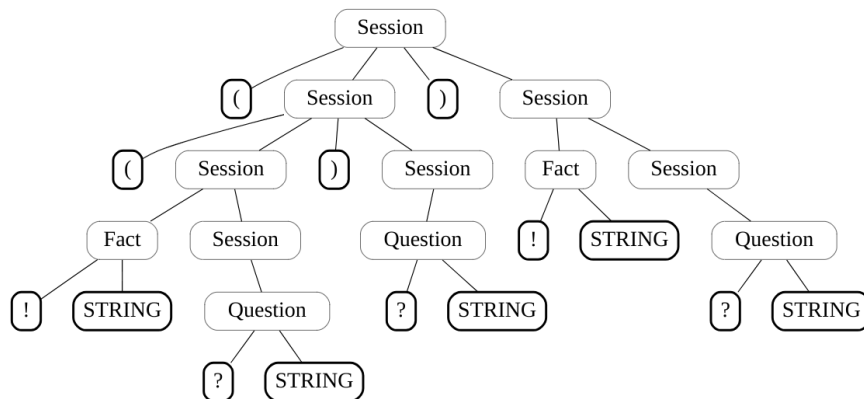
Session  Session -> Fact Session   Session -> Question  Session -> ( Session ) Session
Fact      Fact -> ! STRING
Question   Question -> ? STRING

```

Ora possiamo usare esattamente lo stesso `parse_noε`, passando questa tabella, per derivare una parola

```
INPUT = '( ( ! STRING ? STRING ) ? STRING ) ! STRING ? STRING'.split()
```

```
ProductionGraph(parse_noε(G, TABLE, INPUT))
```



3.6.1 Cosa accade con le epsilon regole

La presenza delle epsilon regole serve a tradurre meglio l'idea semantica che ho in testa, una sessione di colloquio con il mio sistema esperto è una sequenza di fatti, potenzialmente vuota, seguita da una domanda. Per fare questa roba ho bisogno di fare sparire i fatti, quindi ho bisogno di epsilon regole.

```

1 # fig. 8.9, pag. 242
2
3 G = Grammar.from_string("""
4 Session -> Facts Question | ( Session ) Session
5 Facts -> Fact Facts | ε
6 Fact -> ! STRING
7 Question -> ? STRING
8 """)

```

Cominciamo col ragionare per gradi, come facciamo la tabella dei first con le epsilon regole? Quello che potrei fare è fare in modo che table sbirci non solo il primo simbolo ma un po' più in giù nella pila, in realtà vedremo che non è necessario, si potrà pre-computare una tabella.

Occuparsi di first in presenza di epsilon regole non è così devastante. Qui non possiamo più fattorizzare il fatto che dobbiamo per forza prendere la prima lettera perchè non sparisce, non è vero, perchè ora può sparire perchè ci sono le epsilon regole. Non vale più $FIRST(B\beta) = FIRST(B)$. Nel caso $FIRST(B\gamma)$:

- $\epsilon \notin FIRST(B)$ allora $FIRST(B\gamma) = FIRST(B)$ sono nel caso di prima
- $\epsilon \in FIRST(B)$ allora $FIRST(B\gamma) = (FIRST(B) - \epsilon) + FIRST(\gamma)$, dove gamma sono i simboli che seguono B. Il dramma quindi è che devo capire cosa succede a gamma.

Faccio una chiusura, first la computo per tutte le forme sentenziali, quindi una entry della tabella è una tupla: Riempio mettendo first smontando la forma sentenziale tranne la nullabilità il prefisso e se c'è la nullabilità il suffisso. Consideriamo anche la stringa vuota e il simbolo di fine nastro(per praticità, considereremo anche il caso di suffissi di lunghezza nulla, a cui corrisponderà ϵ). Ad ogni passo la chiusura, per ciascun suffisso $\omega = C\delta$, aggiungerà all'insieme $First(\omega)$ i terminali in:

- $First(C)$ se $\epsilon \notin First(C)$
- $First(C) - \epsilon$ e $First(\delta)$ se $\epsilon \in First(C)$

Per concludere, la seguente implementazione popola FIRST con $FIRST(\omega)$ per tutti i suffissi dei lati destri delle produzioni di G anche qualora siano presneti epsilon regole:

```

1  def compute_efirst(G):
2
3      FIRST = Table(1, element = set) # questo significa che gli elementi
4          dell tabella sono insiemi
5
6      # i casi base
7      for t in G.T | {e, HASH}: FIRST[(t,)] = {t} # attenzione, gli indici
8          sono forme sentenziali, ossia tuple!
9
10     # il caso di forma sentenziale di lunghezza nulla
11     FIRST[tuple()] = {e}
12
13     @closure
14     def update_with_suffixes(FIRST):
15         for A, a in G.P:
16             FIRST[(A,)] |= FIRST[a]
17             for o in suffixes(a):
18                 C, *d = o
19                 FIRST[o] |= FIRST[(C,)] - {e}
20                 if e in FIRST[(C, )]: FIRST[o] |= FIRST[d]
21         return FIRST
22
23     return update_with_suffixes(FIRST)

```

Chiamando questa funzione sulla nostra grammatica otteniamo:

Session	$\{ (, !, ? \}$
Fact	$\{ ! \}$
Question	$\{ ? \}$
Facts	$\{ !, e \}$

3.7 Full LL(1)

Dobbiamo vedere se riusciamo ad usare questa cosa del First per fare il parsing, quello che mi interessa veramente è essere in un caso deterministico, quindi in qualche modo posso costruire la tabella a patto di poter calcolare questo $FIRST(\alpha\gamma\#)$. In buona sostanza quello che vorremmo e' calcolare TABLE in funzione di FIRST.

Vediamo che per fare questo ci costruiamo una funzione, partendo da quella dell'altra volta che calcola la tabella dobbiamo estenderla per fare questo calcolo su una forma sentenziale, la chiusura funziona bene se il numero di passi è finito, ma qui non è noto a priori perchè parto da una lunghezza arbitraria, quindi l'unico modo di fare questa cosa è la ricorsione e la logica è questa:

- se la forma sentenziale è lunga uno ritorniamo una epsilon
- se c'è qualcosa da mangiare vuol dire che c'è un C e qualcos'altro. Tutto quello che viene da C lo devo prendere (perchè è il first), tranne epsilon, e poi devo vedere se C è nullabile, se lo è allora devo vedere il suffisso e quindi calcolare il first del suffisso.

```

1 def make_first_function(G):
2
3     FIRST = compute_first(G)
4
5     @show_calls(True)
6     def FIRSTf(w):
7         if not w: return {e} # gestisce il caso provocato dalle
8             fattorizzazioni con |w| = 1, ossia |d| = 0
9         C, *d = w
10        if e in FIRST[(C, )]:
11            return (FIRST[(C, )] - {e}) | FIRSTf(d)
12        else:
13            return FIRST[(C, )]
14    return FIRSTf

```

Quindi adesso posto che siamo in grado di calcolare questo FIRST posso adoperare la costruzione della tabella con la stessa logica, quindi a questo punto avrei bisogno di una tabella (che non è più una tabella) che data la lettera a mi dice se posso sostituire quella produzione. La posso sostituire guardando tutta la pila, devo in qualche modo considerare tutto la pila, non posso guardare solo la cima proprio perchè se il top sparisce sono fregato. Quindi di nuovo voglio lavorare sulla costruzione della tabella ragionando sulla possibilità che la cima sparisca, questo modifica radicalmente l'algoritmo, perchè non usa più una pila. Come funziona la costruzione della tabella? dato che la pila sta in quelle condizioni le possibili produzioni sono queste: devo smontare la pila in A, gamma e cancelletto $A, \gamma, \#$, lo smonto e vado a prendere tutte le alternative, faccio la forma sentenziale e prendo tutte le alternative di a e se trovo una produzione che è in first della

forma sentenziale. Una volta che ho testato tutte le alternative possono succedere tre cose:

- Non ne ho trovata nessuna
- Ne ho trovate più di una: non ho più il determinismo, quindi non posso costruire la tabella
- Ne ho trovata una sola: allora posso costruire la tabella

La costruzione non è precomputabile e dipende da un FIRST che deve guardarsi tutta la pila ed è importante notare che in questo caso a differenza di prima non possiamo scartare delle grammatiche, ma capiamo se qualcosa non va solo a runtime.

```
1 def make_table_function(G):
2
3     FIRSTf = make_first_function(G)
4
5     def TABLEf(stack, a):
6         prods = set()
7         A, *Ts = list(stack) # AT#
8         for a in G.alternatives(A):
9             w = a + tuple(Ts) # aT#
10            if a in FIRSTf(w): prods.add(Production(A, a))
11            if len(prods) == 1:
12                return prods.pop()
13            elif len(prods) > 1:
14                warn(f'Conflict {prods=}')
15            return None
16
17     return TABLEf
```

Vediamo anche il parse, che è molto simile al parse no epsilon, con la differenza che non vedo una tabella precomputata ma invoco una funzione che ha il binocolo sullo stack. Le principali differenze sono:

- ignorare le epsilon aggiunte in pila (per via delle epsilon regole)
- usare la funzione TABLEf per calcolare la produzione da usare (invece di consultare la tabella TABLE)

```
1 def full_parse(G, TABLEf, INPUT):
2     tid = TopDownInstantaneousDescription(G, INPUT)
3     while not tid.is_done():
4         if tid.top() in G.N:
5             P = TABLEf(tid.stack, tid.head())
6             if P is None:
7                 warn(f'No production for ({tid.top()}, {tid.head()}) at
8                     {tid.steps}')
9                 return None
10            tid = tid.predict(P)
11        else:
12            if tid.top() == tid.head():
13                tid = tid.match()
14            else:
```

```

14     warn(f'Expecting {tid.top()}, got {tid.head()}')
15     return None
16 return Derivation(G).leftmost(tid.steps)

```

Il punto cruciale è che purtroppo questa cosa $FIRST(\alpha\gamma\#)$ sembrerebbe non precomputabile. Un modo per risolvere un infinito come questo è quello di chiederci se vogliamo risolverlo in ogni circostanza, in questo caso noi sappiamo che se epsilon non fa parte di $FIRST(\alpha)$ allora il FIRST di tutto quello prima è uguale a $FIRST(\alpha) - \epsilon$ (il caso in cui non sparisce alpha) se invece sparisce abbiamo un po' di cosa che produco dalla tabella e la cosa che mi fa veramente male $FIRST(\alpha) - \epsilon \cup FIRST(\gamma\#)$. Quello che vorremmo calcolare è la funzione FOLLOW(A) che sono l'insieme dei terminali che mi porta ad avere $S \rightarrow xA\gamma \rightarrow xAt\gamma\#$. Stiamo dicendo che per scegliere una produzione A produce alpha guardo:

- i caratteri che produce FIRST(alpha)
- Se alpha sparisce allora guardo i caratteri dopo A, nel nostro caso usando FOLLOW(A)

Il dubbio è se questa cosa la possiamo precomputare. Andiamo per passi, vediamo prima se riusciamo a costruire la tabella, se riesco a costruire la tabella finita abbiamo ricostruito il parser. La calcoliamo così, guardo le letterine dentro FIRST se non c'è epsilon ho finito, se c'è devo guardare le letterine dentro FOLLOW. Quello che ci interessa è non avere conflitto nella tabella, adesso vorremmo distinguere due casi nei conflitti:

- Conflitto FIRST FIRST: due produzioni che hanno la stessa letterina in prima posizione
- Conflitto FIRST FOLLOW: due produzioni che hanno la stessa letterina in prima posizione e una delle due ha epsilon
- Conflitto FOLLOW FOLLOW: due produzioni che hanno la stessa letterina in prima posizione e una delle due ha epsilon

Quindi nella costruzione della tabella mi tengo delle tabelle intermedie giusto per sapere devo è avvenuto il conflitto. Provo tutti i non epsilon e li metto nella tabella, se c'è un epsilon provo tutti i Follow e li metto nella tabella (a parte la gestione degli errori)

```

1 def compute_table(G, FIRST, FOLLOW):
2
3     TABLE = Table(2)
4     FIRST_TABLE = Table(2)
5     FOLLOW_TABLE = Table(2)
6
7     for P in G.P:
8         A, alpha = P
9         for a in FIRST[alpha] - {epsilon}:
10             if FIRST_TABLE[A, a] is not None:
11                 warn(f'First/first conflict on ({A}, {a}) for production {P}
12                     (was {FIRST_TABLE[A, a]})')
13             else:
14                 FIRST_TABLE[A, a] = P
15                 TABLE[A, a] = P

```

```

15     if epsilon in FIRST[alpha]:
16         for a in FOLLOW[A]:
17             if FIRST_TABLE[A, a] is not None:
18                 warn(f'First/follow conflict on ({A}, {a}) for production {P}
19                     (was {FIRST_TABLE[A, a]})')
20             if FOLLOW_TABLE[A, a] is not None:
21                 warn(f'Follow/follow conflict on ({A}, {a}) for production
22                     {P} (was {FOLLOW_TABLE[A, a]})')
23             if FIRST_TABLE[A, a] is None and FOLLOW_TABLE[A, a] is None:
24                 FOLLOW_TABLE[A, a] = P
25                 TABLE[A, a] = P
26         return TABLE

```

Se la tabella contiene al più una produzione per ogni cella, allora la grammatica è detta strong LL(1). Osservate che se una grammatica è full LL(1) allora è anche strong LL(1).

Vediamo come computare FOLLOW, prendo questa produzione $z \rightarrow \alpha A \beta$ e la uso per calcolare $FOLLOW(A)$. Per stabilire cosa c'è nel $FOLLOW(A)$ devo guardare tutte le possibili fattorizzazioni, quindi ci fa senz'altro i $FIRST(\beta)$ ma il dramma è sempre il solito, se beta sparisce cosa ci metto dentro? se sparisce ci sarà dentro banalmente $FOLLOW(x)$. Per costruire chi segue un non terminale A, devo considerare i lati sinistri delle produzioni, parto da FIRST di beta ma se questo sparisce semplicemente un lato sinistro di A sarà x. Il follow di un terminale non mi interessa, non avrebbe senso.

```

1 def compute_follow(G, FIRST):
2
3     FOLLOW = Table(1, set)
4     FOLLOW[G.S] = {HASH}
5
6     @closure
7     def complete_follow(FOLLOW):
8         for X, w in G.P:
9             for Y in suffixes(w): # X -> alpha A Beta (alpha e' mangiato dal
10                 suffisso)
11                 A, *Beta = Y
12                 if A in G.T: continue
13                 FOLLOW[A] |= FIRST[Beta] - {epsilon}
14                 if epsilon in FIRST[Beta]: FOLLOW[A] |= FOLLOW[X]
15         return FOLLOW
16
17     return complete_follow(FOLLOW)

```

Vediamo questo con l'esempio dei FACTS and QUESTION, quella con le epsilon regole. La grammatica è questa:

```

1 G = Grammar.from_string("""
2 Session -> Facts Question | ( Session ) Session
3 Facts -> Fact Facts | epsilon
4 Fact -> ! STRING
5 Question -> ? STRING
6 """)

```

Precomputiamo prima le tabelle first e follow:

```
FIRST = compute_first(G)
FIRST.restrict_to({(N, ) for N in G.N})
```

Facts !, ε

Fact !

Question ?

Session !, (, ?

```
FOLLOW = compute_follow(G, FIRST)
FOLLOW
```

Session #,)

Facts ?

Question #,)

Fact !, ?

ε ?

Possiamo quindi costruire la tabella per la grammatica:

```
TABLE = compute_table(G, FIRST, FOLLOW)
TABLE
```

!

?

(

Session Session -> Facts Question Session -> Facts Question Session -> (Session) Session

Facts Facts -> Fact Facts Facts -> ε

Fact Fact -> ! STRING

Question Question -> ? STRING

A questo punto vediamo il parser che è tornato banale perchè guardiamo la tabella precomputata (è identico al parser senza epsilon regole), l'unica cosa che cambia è la costruzione della tabella e il calcolo del follow.

```
1 def parse(G, TABLE, INPUT):
2     tid = TopDownInstantaneousDescription(G, INPUT)
3     while not tid.is_done():
4         if tid.top() in G.N:
5             P = TABLE[tid.top(), tid.head()]
6             if P is None:
7                 warn(f'No production for ({tid.top()}, {tid.head()}) at
                        {tid.steps}')
8                 return None
9             tid = tid.predict(P)
10        else:
11            if tid.top() == tid.head():
12                tid = tid.match()
13            else:
14                warn(f'Expecting {tid.top()}, got {tid.head()}')
15                return None
```

```
16 return Derivation(G).leftmost(tid.steps)
```

3.7.1 Conflitti

Vediamo i vari casi di conflitto attraverso degli esempi. Introduciamo una funzione di comodo che faccia i vari passi di computazione:

```
1 def fft(G):
2     FIRST = compute_first(G)
3     FOLLOW = compute_follow(G, FIRST)
4     FF = Table(2)
5     for N in G.N:
6         FF[N, 'First'] = FIRST[(N, )]
7         FF[N, 'Follow'] = FOLLOW[N]
8     return FF, compute_table(G, FIRST, FOLLOW)
```

Caso first/first

```
G = Grammar.from_string("""
S -> a | A
A -> a
""")

FF, TABLE = fft(G)

First/first conflict on (S, a) for production S -> A (was S -> a)

FF

    First Follow
S   a   ε
A   a   ε

TABLE

    a
S S -> a
A A -> a
```

Caso first/follow

```
G = Grammar.from_string("""
S -> A a
A -> a | ε
""")

FF, TABLE = fft(G)

First/follow conflict on (A, a) for production A -> ε (was A -> a)

FF

    First Follow
S   a   ε
A  a, ε   a

TABLE

    a
S S -> A a
A  A -> a
```

Caso follow/follow

```
G = Grammar.from_string("""
S -> A a
A -> B | C
B -> ε | b
C -> ε | c
""")

FF, TABLE = fft(G)

Follow/follow conflict on (A, a) for production A -> C (was A -> B)

FF

First Follow
B b, ε a
S a, b, c ε
C c, ε a
A ε, b, c a

TABLE

      c      a      b
S S -> A a S -> A a S -> A a
A A -> C   A -> B   A -> B
B      B -> ε   B -> b
C C -> c   C -> ε
```

3.7.2 Riparare i conflitti

Riparare i conflitti è facile, l'idea è questa. Le cose vanno male quando succedono delle cose così $A \rightarrow \alpha\beta|\alpha\gamma$, si rompe quando due produzioni cominciano con la stessa cosa. Possiamo facilmente notare che è impossibile in questo caso decidere che produzione prendere sulla base di $FIRST(\alpha)$. Una tecnica abbastanza banale per risolvere questi problemi è la fattorizzazione, mediante l'introduzione di un nuovo terminale N possiamo cambiare le due produzioni e sperare che $FIRST(\beta)$ e $FIRST(\gamma)$ siano disgiunti.

$$N \rightarrow \beta|\gamma$$

$$A \rightarrow \alpha N$$

Questa cosa non risolve sempre perchè se beta e gamma hanno un intersezione non vuota sono a capo, ma spero di poter differenziare i simboli.

Proviamo ad usare, come esempio, una grammatica per operazioni aritmetiche in cui ogni operatore coinvolga esattamente due sottoespressioni:

```
1 G = Grammar.from_string("""
2 E -> T + T | T
3 T -> F * F | F
4 F -> ( E ) | i
5 """)
```

Troviamo dei conflitti:

FF, TABLE = fft(G)

First/first conflict on (E, i) for production E -> T (was E -> T + T)
 First/first conflict on (E, () for production E -> T (was E -> T + T)
 First/first conflict on (T, i) for production T -> F (was T -> F * F)
 First/first conflict on (T, () for production T -> F (was T -> F * F)

FF

	First	Follow
F	i, (*, \$, +,)	
E	i, (\$,)	
T	i, (\$, +,)	

TABLE

	i	(
E	E -> T + T	E -> T + T
T	T -> F * F	T -> F * F
F	F -> i	F -> (E)

Che possono essere risolti fattorizzando sia T che F:

```
1 G = Grammar.from_string("""
2 E -> T E'
3 E' -> + T | epsilon
4 T -> F F'
5 F' -> * F | epsilon
6 F -> ( E ) | i
7 """)
```

FF, TABLE = fft(G)

TABLE

i (+ #) *

E E -> T E' E -> T E'

E' E' -> + T E' -> E E' -> E

T T -> F F' T -> F F'

F' F' -> E F' -> E F' -> E F' -> * F

F F -> i F -> (E)

3.8 Parser ricorsivo discendente predittivo

Scriviamo una famiglia di funzioni ricorsive ed una delle quali effettuerà il parsing del documento. Quello che ci aveva bloccato l'altra volta era che non sapevamo cosa fare quando c'erano due regole, prima prendevamo solo la prima possibilità, però non è detto che la prima scelta sia sempre quella giusta. La soluzione a questo problema è avere un parser ricorsivo discendente predittivo, abbiamo messo l'automa nelle condizioni di guardare il nastro e la pila, e avendo la tabella possiamo decidere cosa fare. Prendiamo in mano la TABLE e la adopero per costruire un parser ricorsivo discendente che quando deve decidere cosa fare guarda la tabella e non prende per forza la prima alternativa.

I passi che faremo sono:

- Costruzione manuale del riconoscitore
- Costruzione del parse che restituisca i passi di derivazione

- Costruzione automatica del parser (una funzione che crea la funzione parser)
- Costruire un interprete

Una cosa molto interessante nel parser topdown è che si può mischiare il parsing al riconoscimento, potrei mano a mano che mangio l'input calcolare l'espressione (nel caso di grammatica aritmetica), possiamo quindi mischiare il processo di parse con quello di interpretazione. Nel caso dell'interprete è cruciale sciogliere i terminali generici (i -> qui ad un certo punto ci sarà un numero). Dovremo rendere più esplicito il processo di parsing e tokenizzazione.

Partiamo da una grammatica per le espressioni aritmetiche:

```
1 G = Grammar.from_string("""
2 Expr -> Term Expr'
3 Expr' -> PLUS Term Expr' | epsilon
4 Term -> Factor Factor'
5 Factor' -> TIMES Factor Factor' | epsilon
6 Factor -> LPAR Expr RPAR | NUMBER
7 """)
```

Con questa grammatica possiamo costruire la tabella:

```
FIRST = compute_first(G)
FOLLOW = compute_follow(G, FIRST)
compute_table(G, FIRST, FOLLOW)
```

	NUMBER	LPAR	PLUS	RPAR	#	TIMES
Expr	Expr -> Term Expr'	Expr -> Term Expr'				
Expr'			Expr' -> PLUS Term Expr'	Expr' -> ε	Expr' -> ε	
Term	Term -> Factor Factor'	Term -> Factor Factor'				
Factor'			Factor' -> ε	Factor' -> ε	Factor' -> ε	Factor' -> TIMES Factor Factor'
Factor	Factor -> NUMBER	Factor -> LPAR Expr RPAR				

Data questa tabella mi costruisco le funzioni parse per ogni riga della tabella, ogni funzione è costruita sulla base di quello che c'è nella riga della tabella (se incontri questo usa questa produzione), consideriamo che nella tabella gli spazi vuoti sono quelli di errore, non ci sono produzioni e devo segnalare l'errore. Riusciamo a distinguere tra le colonne utilizzando il look-ahead.

```
1 def parse_Expr():
2     if token in {'LPAR', 'NUMBER'}:
3         parse_Term()
4         parse_Exprp()
5     else:
6         warn('Error parsing Expr')
7
8 def parse_Exprp():
9     if token in {'RPAR', HASH}:
10         pass # questa e' l'epsilon-produzione
11     elif token == 'PLUS':
12         consume('PLUS')
13         parse_Term()
```

```

14     parse_Exprp()
15 else:
16     warn('Error parsing Expr_primo')
17
18 def parse_Term():
19     if token in {'LPAR', 'NUMBER'}:
20         parse_Factor()
21         parse_Factorp()
22     else:
23         warn('Error parsing Term')
24
25 def parse_Factorp():
26     if token in {'PLUS', 'RPAR', HASH}:
27         pass
28     elif token == 'TIMES':
29         consume('TIMES')
30         parse_Factor()
31         parse_Factorp()
32     else:
33         warn('Error parsing Factor_primo')
34
35 def parse_Factor():
36     if token == 'NUMBER':
37         consume('NUMBER')
38     elif token == 'LPAR':
39         consume('LPAR')
40         parse_Expr()
41         consume('RPAR')
42     else:
43         warn('Error parsing Factor')

```

Usa l'uguaglianza con il token (che al momento è un mistero ma rappresenta quello che c'è sul nastro) e in base a questa uguaglianza decide se fare una produzione o meno, se ho delle epsilon regole non faccio niente, altrimenti usano il lato destro, se c'è un terminale lo consumano e se c'è un non terminale fanno il parser di quello.

3.8.1 Tokenizzazione

Ci manca di capire cosa fa la funzione consume e il tokenizzatore. Ci piacerebbe molto ragionare in questo modo, l'input è una sequenza di caratteri e le cose che adopero per descrivere la grammatica sono oggetti più sofisticati. Nella libreria che stiamo usando stiamo usando le str di Python che hanno un brutto difetto, non esistono i caratteri (ci sono eventualmente stringhe lunghe 1). Noi stiamo usando come parole del linguaggio le tuple di simboli (o stringhe lunghe 1).

- Simboli -> stringhe
- Parole o forme sentenziali -> tuple di simboli
- Input -> sequenza di simboli -> tuple

Quello su cui dobbiamo ragionare è come trasformare le sentenze della grammatica e trasformarla correttamente in una tupla formattata correttamente per il parser. Abbiamo quindi bisogno di un processo che prenda la forma sentenziale, la trasformi in una

opportuna forma che il parser possa adoperare:

$$'34 + 2' \rightarrow ('3', '4', '+', '2')$$

Questa cosa si chiama TOKENIZZAZIONE, che ha l'obiettivo di prendere una sequenza di caratteri di un alfabeto (tipo ASCII) e trasformarla in una sequenza di simboli.

Abbiamo una prima grammatica che è quella di tokenizzazione, è quella che vede la stringa vera e propria e ha un insieme di terminali che si chiama alfabeto di macchina (la roba che sta scritta dentro il file). Quella grammatica lì è una collezione di linguaggi regolari, l'idea è che voglio frazionare la stringa usando una serie di grammatiche regolari. Teniamoci divisi i non terminali e le produzioni, distinguiamo i simboli distinti, tanti linguaggi regolari e il simbolo distinto è il nome di quello che vediamo.

I terminali del parser sono i simboli distinti del tokenizzatore. Il parser lavora dalle teste degli alberi del tokenizzatore e il tokenizzatore lavora dalle stringhe alle teste degli alberi del tokenizzatore. Il tokenizzatore associa anche ad ogni token il suo value (il token è il simbolo distinto e il value è il valore che ha), ad esempio un token NUMBER può avere un valore 37. Inoltre deve essere in grado di fare scorrere il puntatore tra i non terminali solo nel caso in cui la produzione sia corretta.

Quello che vedremo è quella che fa anche ANTLR, che useremo, produce una prima grammatica che è quella di tokenizzazione e poi per ciascun non terminale produce tramite espressioni regolari una grammatica che si mangia i simboli distinti e produce i token. La grammatica di tokenizzazione la rappresentiamo come un unione di espressioni regolari:

```
1 EPXR_TOKEN2PATTERN = (  
2   ('LPAR',    r'\('),  
3   ('RPAR',    r'\)'),  
4   ('PLUS',    r'\+'),  
5   ('TIMES',   r'\*'),  
6   ('NUMBER',  r'\d+'),  
7 )
```

Adesso devo operativamente usare questa cosa per ottenere il token ed il valore. Abbiamo una funzione consume che consuma il token e il valore, quindi la funzione consume è una funzione che mi dice se il token che ho in testa è quello che mi aspetto, se è così lo consumo e lo restituisco. Se non è così mi dà errore. Faccio quello che abbiamo visto con le espressioni regolari, metto in or tutte le espressioni regolari e tra quegli or avrò a non null solo il simbolo che ho matchato. Quando ha finito di iterare invece di lanciare un'eccezione restituisce un token speciale di tappo. Da notare che usiamo due variabili globali token e value per immagazzinare il look-ahead:

```
1 def make_consume(word, token2pattern):  
2     tokens_re = re.compile('|'.join(f'(?P<{token}>{pattern})' for token,  
3                                     pattern in token2pattern))  
4     token_value_iterator = iter((token, value) for match in  
5                                 tokens_re.finditer(word) for token, value in  
6                                 match.groupdict().items() if value)  
7     def _advance():  
8         global token, value  
9         token, value = next(token_value_iterator, (HASH, None))  
10    _advance()  
11    def consume(expected):  
12        if token != expected:
```

```

10     warn('Expected {}, found {}'.format(expected, token))
11     else:
12         _advance()
13     return consume

```

3.8.2 Interprete, valutazione delle espressioni

Usando il parser posso tornare indietro con il tokenizzatore usando i valori per tirare fuori il valore dell'espressione. Costruiamo delle espressioni ricorsive per cui per ogni terminale mi danno il valore dell'espressione.

```

1  def eval_Expr():
2      if token in {'LPAR', 'NUMBER'}:
3          val = eval_Term()
4          val += eval_Exprp()
5          return val
6      warn('Error parsing Expr')
7
8  def eval_Exprp():
9      if token in {'RPAR', HASH}:
10         return 0 # questa e' l'unita addittiva
11     elif token == 'PLUS':
12         consume('PLUS')
13         val = eval_Term()
14         val += eval_Exprp()
15         return val
16     warn('Error parsing Expr_primo')
17
18  def eval_Term():
19      if token in {'LPAR', 'NUMBER'}:
20         val = eval_Factor()
21         val *= eval_Factorp()
22         return val
23     warn('Error parsing Term')
24
25  def eval_Factorp():
26      if token in {'PLUS', 'RPAR', HASH}:
27         return 1 # questa e l'unita moltiplicativa
28     elif token == 'TIMES':
29         consume('TIMES')
30         val = eval_Factor()
31         val *= eval_Factorp()
32         return val
33     warn('Error parsing Factor_primo')
34
35  def eval_Factor():
36      if token == 'NUMBER':
37         val = int(value) # la variabile globale modificata da consume
38         consume('NUMBER')
39         return val
40     elif token == 'LPAR':
41         consume('LPAR')
42         val = eval_Expr()
43         consume('RPAR')

```

```

44     return val
45     warn('Error parsing Factor')

```

Posso estendere la grammatica per fare in modo che il parser prenda anche delle espressioni regolari, in particolar modo con l'asterisco indichiamo che possiamo mettere N termini come quello. Nelle funzioni indichiamo questa cosa con un while, quindi se ho un asterisco lo metto dentro un while, devo però stare attento a quando devo fermarmi dentro il while:

```

1  def eval_Expr():
2      if token in {'LPAR', 'NUMBER'}:
3          val = eval_Term()
4          while token == 'PLUS':
5              consume('PLUS')
6              val += eval_Term()
7          if token in {'RPAR', HASH}: return val
8          warn('Error parsing Expr')
9
10 def eval_Term():
11     if token in {'LPAR', 'NUMBER'}:
12         val = eval_Factor()
13         while token == 'TIMES':
14             consume('TIMES')
15             val *= eval_Factor()
16         if token in {'PLUS', 'RPAR', HASH}: return val
17         warn('Error parsing Term')

```

3.9 Parsing deterministico bottom-up LR(0)

Ci serve un modo per eliminare il non determinismo, in questo caso non possiamo usare la tabella perche' qui non guardiamo neanche il nastro per scegliere se fare shift o reduce. Useremo un automa a pila. L'idea e' che la pila sara' sia la sequenza di stati dell'automa, quindi maneggiare la pila sara' come maneggiare con gli stati dell'automa, con la cosa bella che togliere le cose dalla pila significa andare indietro nella storia dell'automa.

3.9.1 Automa di KNOT

La cosa cruciale e' come costruire questo automa. Usa degli accorgimenti molto simili a quando ci siamo convinti della regolarita' del linguaggio, qui per fare questa costruzione si usa storicamente un'altra rappresentazione rispetto a quella che avevamo usato.

Item

Si costruisce a partire da una cosa che si chiama ITEM, un item e' una produzione annotata in cui scelgo un modo per indicare quanta parte della produzione ho mangiato con un pallino che indica che la parte a sinistra del pallino e' stata mangiata e la parte a destra del pallino e' quella che devo ancora mangiare $A \rightarrow \alpha \cdot \beta$. Questi oggetti saranno usati per rappresentare la computazione dell'automa che ci dice se dobbiamo fare shift o reduce. Per usare questi item dobbiamo riflettere su due aspetti:

- Come si passa da un item all'altro
- Vorremmo arrivare a compimento un automa deterministico. Sappiamo che si determina con la power construction. Noi ragioneremo con una epsilon closure.

Una cosa cruciale dell'item e' che quel pallino si sposta verso destra (ogni volta che consumo una regola) e' il superamento di un simbolo che puo' fare sfruttando un terminale o un non terminale ed in ogni caso quello che mi interessa e' quello che viene dopo il pallino perche' e' la modifica che posso apportare al mio item, possiamo trovarci anche nel caso in cui il pallino e' alla fine della produzione, significa che quella produzione l'ho usata tutta.

La libreria Liblet contiene un po' di questa zuppa, partiamo da una grammatica MCD:

```
1 G = Grammar.from_string("""
2 E -> E - T | T
3 T -> n | ( E )
4 """)
```

la libreria ci permette di costruire un item a partire da una produzione, e possiamo dirgli di avanzare o meno:

```
1 # Si costruisce dato rhs e lhs (come le produzioni)
2
3 A, alpha = G.P[0]
4
5 item = Item(A, alpha)
6
7 item
8 E -> .E - T
9 # ha un metodo advance che sposta il punto e un metodo
10 # symbol_after_dot che riporta il simbolo che segue il
11 # punto (o None se il punto e' oltre la fine del rhs)
12
13 for X in alpha:
14     item = item.advance(X)
15     print(f'item = {item}, symbol_after_dot = {item.symbol_after_dot()}')
16 item = E -> E.-T, symbol_after_dot = -
17 item = E -> E - .T, symbol_after_dot = T
18 item = E -> E - T ., symbol_after_dot = None
```

L'idea di questo item e' che rappresenta un atto di derivazione, se mi trovo a sinistra di un simbolo non terminale potrei proseguire oppure cominciare a consumare una qualunque delle alternative che ha quel simbolo a sinistra. Se ho questo item $A \rightarrow \alpha \cdot B\beta$ andando avanti mi trovo a sinistra di un non terminale, che significa trovarsi a sinistra di tutte le sue alternative. L'idea della **epsilon chiusura** e' che ogni volta che il pallino e' a sinistra di un non terminale mi trovo nella stessa condizione in cui sono a sinistra di una qualsiasi sua alternativa $B_n \rightarrow \cdot\beta$. Per ogni item $A \rightarrow \alpha \cdot B\beta$ mettiamo in un set tutte le alternative di $B \rightarrow \cdot\gamma$ dove $B \rightarrow \gamma$ e' una produzione di B nella grammatica G. Trovarsi davanti ad un non terminale significa chiedersi che sua alternativa adoperare, mettiamo tutte queste alternative in un set, che ci porta ad avere degli item set. Questo e' il codice di questa idea:

```
1 @closure
2 def epsilon_closure(items, G):
```

```

3   added = set()
4   for item in items:
5       X = item.symbol_after_dot()
6       if X is None or X in G.T: continue
7       added |= {Item(X, Beta) for Beta in G.alternatives(X)}
8   return frozenset(items | added)
9   # vediamo come si comporta sul set di item che contiene solo l'item
10  # ottenuto dalla prima produzione
11
12  set(epsilon_closure({Item(*G.P[0])}, G))
13  {E -> .E - T, E -> .T, T -> .(E), T -> .n}

```

Costruzione dell'automa

Ora vogliamo costruire l'automa, prima abbiamo costruito gli stati (rappresentano la possibile circostanza del nostro parsing)

Per noi lo stato iniziale dell'automa e' l'epsilon chiusura di S, il pallino a sinistra di tutte le alternative di S.

La funzione di transizione di questo automa e' la seguente, posso spostarmi da uno stato ai suoi successivi transendo a partire da qualunque simbolo, prendo un elemento dell'item set, chiamo la funzione advance che mi fa andare avanti e questo mi portera' ad avere altri item set che poi chiudo:

```

1  def delta(items, X, G):
2      return epsilon_closure({item.advance(X) for item in items if
                               item.advance(X)}, G)

```

Gli stati finali dell'automa, nell'idea iniziale che la condizione che ci piace e' quando terminiamo una produzione, saranno gli item set in cui sono in queste condizioni $A \rightarrow \alpha \cdot$, ho una sola alternativa di sostituzione.

Costruzione del parser

Parto da q_0 , poi ho una transizione x che mi porta ad altri stati, li provo tutti, faccio una nuova transizione e vado avanti cosi'. Sostanzialmente e' una visita in ampiezza, ogni visita mi creera' dei nuovi insiemi e vado avanti fino a che non trovo uno stato finale. Da notare che mettiamo in coda alla grammatica un tappo, cosi' da sapere quando abbiamo finito di mangiare che mi evita di essere in una situazione in cui l'automa mi dica che ha finito quando non e' vero.

L'algoritmo e' la solita visita in ampiezza, mi tengo tutte le transizioni e gli stati e le provo tutte, sposta il pallino e se vede un non terminale ci mette dentro tutte le alternative:

```

1  def compute_lr0_automaton(G):
2
3      states = {q0(G)}
4      transitions = []
5      q = Queue([q0(G)])
6      while q:
7          items = q.dequeue()
8          for X in G.T | G.N:
9              next_items = delta(items, X, G)

```

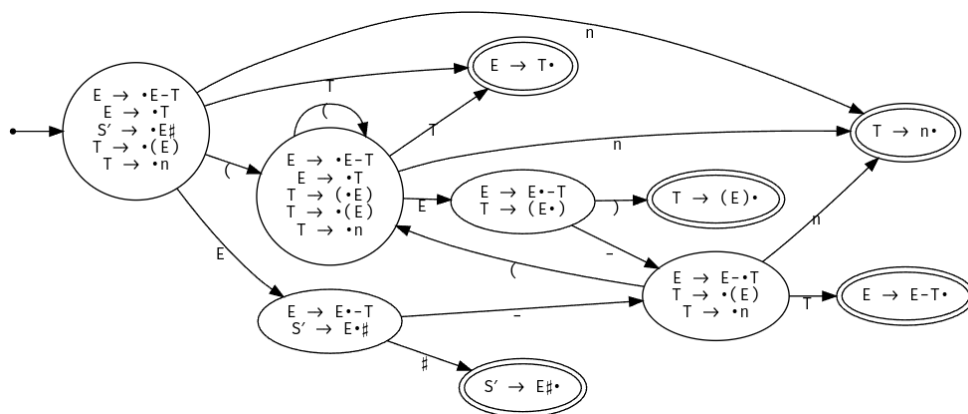
```

10     if not next_items: continue
11     transitions.append(Transition(items, X, next_items))
12     if next_items not in states:
13         states.add(next_items)
14         q.enqueue(next_items)
15
16     return Automaton(
17         states,
18         G.T | G.N,
19         transitions,
20         q0(G),
21         set(state for state in states if len(state) == 1 and
22             first(state).symbol_after_dot() is None)

```

Essattamente come la costruzione della tabella, la costruzione dell'automa e' l'elemento discriminante che ci dice se la grammatica e' LR(0) o meno. Se la costruzione dell'automa termina con uno stato finale allora la grammatica e' LR(0), altrimenti non lo e' (non ci sono conflitti).

Guardando l'automa si capisce che fa ogni scelta di consumare qualcosa, poi quando arriva ad un punto in cui il pallino e' davanti ad un non terminale mette tutte le sue alternative.



Parsing con un automa a pila

L'idea e' che faccio il parsing seguendo l'automa a pila, mi muovo sul grafo e mano mano che mi muovo pusho gli stati sulla pila, quando arrivo ad uno stato finale tolgo quella produzione, torno indietro togliendo quella produzione dallo stato precedente.

Noi guardiamo solo la pila, non sempre necessariamente consumando lo stato che c'e' sulla cima, quindi l'automa spia quello che c'e' sulla pila, e' un po' diverso da un automa a pila. Se la cima della pila e' uno stato finale lo tiro fuori e lo smonto in un lato sinistro, lato destro e la posizione del pallino, significa che ho visto che un non terminale produce solo alpha, quindi so che la riduzione da fare e' quella. Ho sbattuto su uno stato finale, fai la riduzione che c'e' scritta dentro. Adesso devo riavvolgere la computazione facendo tanti pop quanti elementi ci sono nella parte destra della produzione. A questo punto devo scegliere in che direzione andare, siccome ho deciso di fare una riduzione con un non terminale mi sposto avanti seguendo lo stato del non terminale che ho scelto per la

riduzione. Se invece non sono in uno stato finale, vado a vedere nella cima del nastro e lo shift che devo fare e' decisa da quello che c'e' sulla cima della pila. Quando devo fare uno shift guardo quello che ho sul nastro e faccio lo shift seguendo quello che mi dice il nastro. L'idea cruciale e' che adopero la pila per tenere traccia di quello che sto computando. Da notare che produciamo una derivazione right-most, ad ogni passo di consume tiro fuori una regola di derivazione, che se poi leggo da sinistra verso destra ho proprio i passi di derivazione partendo da S.

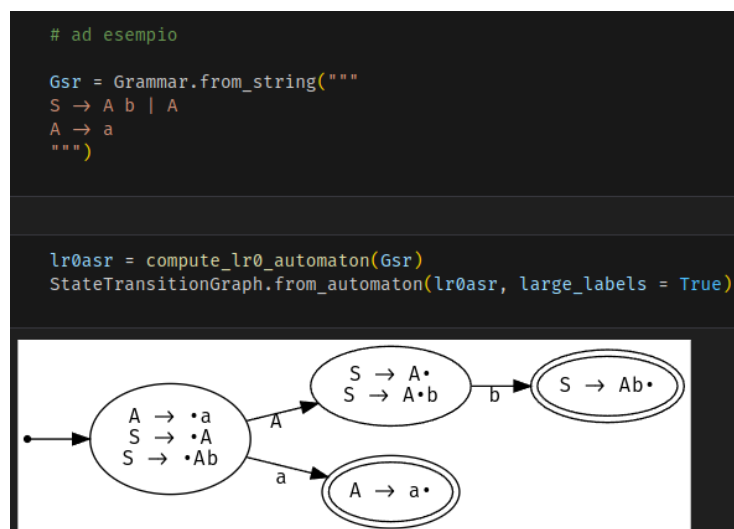
```

1  def lr0parse(lr0a, text, G):
2
3      bid = BottomUpInstantaneousDescription(G, text + [HASH, ])
4      s = Stack([lr0a.q0])
5
6      while True:
7          top = s.peek()
8          if top in lr0a.F: # REDUCE
9              N, alpha, _ = first(top)
10             bid = bid.reduce(Production(N, alpha))
11             if bid.is_done(): return bid.steps
12             for _ in alpha: s.pop()
13             nxt = first(lr0a.delta(s.peek(), N))
14         else: # SHIFT
15             nxt = first(lr0a.delta(top, bid.head()))
16             if nxt is None:
17                 warn(f'Unexpected token {bid.head()}')
18                 return None
19             bid = bid.shift()
20             s.push(nxt)

```

Conflitti

Possiamo vedere degli esempi di grammatiche piu' sfigate per cui il grafo non va bene, la costruzione del grafo mi porta ad avere un conflitto, ho uno stato in cui ho sia un pallino in fondo sia in un pallino che mi porterebbe a fare uno shift e l'automa non sa cosa fare, questo si chiama conflitto shift-reduce.



L'altra circostanza e' questa, sono nel caso di un loop, mi trovo in uno stato in cui posso fare due riduzioni, e' un conflitto reduce-reduce.

```
# ad esempio

Grr = Grammar.from_string("""
S → A | a
A → a
""")

lr0arr = compute_lr0_automaton(Grr)
StateTransitionGraph.from_automaton(lr0arr, large_labels = True)
```

In questo caso è lo stato $\{A \rightarrow a\bullet, S \rightarrow a\bullet\}$ che può essere interpretato come un reduce $A \rightarrow a$ o $S \rightarrow a$.

Osserviamo che non possono mai esserci conflitti tra shift, perche' lo shift e' dato dal carattere sul nastro, non avrebbe senso un conflitto.

I conflitti LR non si vedono altrimenti se non costruire l'automa e vedere se e' rotto, non c'e' modo di vederli dalla grammatica. Cosa succede quando sono in una situazione di conflitto in LL o LR, un conflitto e' uno stato di non determinismo quindi la strada generalmente e' aumentare il look ahead, quindi se sono in una situazione di conflitto shift-reduce posso aumentare il look ahead e vedere se il prossimo simbolo mi da un indizio su cosa fare, LR(N). E' una strategia noiosissima perche' dovrei vedere molti piu' stati.

3.9.2 Tabelle GOTO e ACTIONS

Quello che si fa veramente nella pratica e' codificare l'automa, inizialmente si mettono in una tabella tutti gli stati dell'automa:

```
states = sorted(map(set, lr0a.N))
iter2table(states)
```

0	$\{S' \rightarrow E \# \bullet\}$
1	$\{E \rightarrow E - T \bullet\}$
2	$\{T \rightarrow (E \bullet), E \rightarrow E - T\}$
3	$\{T \rightarrow \bullet n, T \rightarrow \bullet (E), E \rightarrow \bullet E - T, E \rightarrow \bullet T, T \rightarrow \bullet (E)\}$
4	$\{E \rightarrow T \bullet\}$
5	$\{T \rightarrow (E) \bullet\}$
6	$\{T \rightarrow \bullet n, T \rightarrow \bullet (E), E \rightarrow E - T\}$
7	$\{T \rightarrow \bullet n, T \rightarrow \bullet (E), S' \rightarrow \bullet E \#, E \rightarrow \bullet E - T, E \rightarrow \bullet T\}$
8	$\{S' \rightarrow E \# \bullet, E \rightarrow E - T\}$
9	$\{T \rightarrow n \bullet\}$

Se quella tabella ha avuto successo ho anche una mappa per fare le sostituzioni quando devo fare le reduce con le transizioni dell'automa:

```

index_of = states.index

GOTO = Table(2, set)
for frm in states:
    for X in sorted(lr0a.T):
        to = lr0a.d(frm, X)
        if to:
            to = first(to)
            GOTO[index_of(frm), X] = index_of(to)

GOTO

```

	)	-	(	E	T	n	#
2	5	6					
3			3	2	4	9	
6			3		1	9	
7			3	8	4	9	
8	6						0

Poi mi ci vuole la tabella delle azioni, sono nello stato n ma qui cosa faccio shift o reduce, contiene tutti gli stati in cui faccio una reduce ed e' una mappa tra il numero dello stato e la riduzione da fare:

```

ACTIONS = Table(1)

for n, state in enumerate(states):
    if len(state) == 1:
        A, a, _ = first(state)
        ACTIONS[n] = Production(A, a)

ACTIONS

```

0	S' -> E #
1	E -> E - T
4	E -> T
5	T -> (E)
9	T -> n

A questo punto il parser deve tenere il nastro e la pila e quello che devo tenere sulla pila e' la sequenza degli stati, di fatto si puo' provare a scriverlo anche senza le porcate di python che permette di tenere nella pila i set.

Chapter 4

ANTLR

ANTLR e' un parser generator che genera un parser LL top down multi linguaggio. A partire da una grammatica possiamo generare il parser nel linguaggio che vogliamo e ci genera un runtime nel linguaggio che vogliamo per fare funzionare il parser.

La roba interessante e' che ci sono un sacco di grammatiche pronte per ANTLR.

Abbiamo due modi per usare questo strumento:

- Uso diretto di ANTLR, leggendo il manuale e capire come funziona per scrivere il codice python
- Uso mediato da Liblet, significa che il prof ha scritto un po' di codice in Liblet che serve per importare roba di Java in maniera dinamica

Nel progetto e' una delle cose che possiamo scegliere.

4.1 Uso diretto di ANTLR

La cosa principale di questo strumento e' che ci permette di specificare la grammatica context free che vogliamo usare per produrre il parser tramite file o stringa. Questa grammatica e' fatta cosi', definiamo la grammatica, poi ci sono una serie di regole di tokenizzazione scritte in maiuscolo alla cui destra c'e' una espressione regolare. Le regole della grammatica (di parsing) iniziano in minuscolo, sono date da un simbolo di tokenizzazione e poi si possono mettere delle costanti. Alcuni token possono essere scartati dalla parte di parsing.

Adesso il punto cruciale e' utilizzare un tool Java che generi codice Python. Lui genera 4 file:

- lexer
- parser
- visitor: interfaccia che permette di visitare il tree generato dal parser
- listener: interfaccia

Uso del lexer, possiamo vedere i passi del lexer, il token ci fa vedere anche i caratteri del sorgente che rappresentano il token:

Abbiamo visto un esempio piu' complesso di una grammatica di espressioni ed assegnamenti. Vediamo ora come usare il parser, tendenzialmente i parser hanno due modalita' di funzionamento: dato che l'albero e' una struttura ricorsiva ci sono due modi per poterne fruire. Ognuno di questi nodi e' relativo ad una delle regole di parsing, i figli sono i token.

- listener: ha due metodi per ogni regola, entro ed esco. Poi posso invocare un processo di visita che invoca il listener secondo l'ordine che mi aspetto.
- visitor: me lo scrivo io, mi scrivo una funzione per ciascuna regola che sono quelle usate per fare la visita.

Quello che faremo noi a lezione e' usare visitor, perche' a lezione usando LIBLET ci dara' un albero e noi ci scriveremo sempre le visite dell'albero. Così' e' come useremo il listener, creo una classe che implementa il listener, poi creo l'istanza e lo passo al walker di liblet che lo usa per fare la visita:

Nel caso del visitor ho sempre la mia classe che implementa il visitor, mi implemento dentro la visita e poi uso la classe per fare la visita che mi sono scritto:

Fino ad ora non ci siamo mai occupati degli errori, ma in caso di ANTLR si possono specificare degli ascoltatori di errore che hanno un metodo che va invocato ogni volta che lui vede un errore nel lexing o nel parser:

4.2 Uso mediato da Liblet

Il modulo ANTLR dentro Liblet ci permette di prendere una grammatica e mi da un oggetto che ha delle competenze. In questa grammatica ci sono delle etichette di ANTLR che ci servono a distinguere le alternative di una regola che ha delle alternative che e' comodo perche' se avviene un match di stat non sapremmo cosa abbiamo matchato:

Dato l'oggetto possiamo fargli fare diverse cose, inizialmente vediamo i tokens, basta un metodo in questo caso:

Posso avere anche il contesto, che mi da dove partire per il visitor, posso dargli anche il token da cui partire:

L'albero di parsing possiamo produrlo con un metodo tree che produce un tree di liblet con dentro le espressioni:

ANTLR accetta anche grammatiche malate (ricorsivita', precedenze...) c'e' pero' una malattia che non puo' essere riconosciuta a tempo di analisi, la ambiguita'. Si puo' accendere una spia che ci dice se il parsing e' ambiguo, il suggerimento e' che quando si progetta una nuova grammatica si preparano dei testcase e vedo se il diagnostici mi dice se e' la grammatica e' ambigua. Va acceso il diag = True nel context:

4.2.1 Grammatiche note

Possiamo guardare alcune grammatiche note che possono essere utili, ad esempio c'e' una grammatica per l'aritmetica, bella ma l'associativita' a destra la sbaglia. Un'altra grammatica carina e CALCULATOR, che parse le espressioni e permette l'invocazione di funzioni, come il seno o coseno.

Un'altra grammatica super utile, che non viene dal repository standard, ma dal libro di antlr e' la grammatica del linguaggio Cymbol che e' una specie di linguaggio di programmazione che ha l'invocazione a funzione e gli statement.

La fine di questa cosa e' che e' molto potente, e se incontriamo errori guardiamo in documentazione perche' ci sono alcune porcherie.

4.2.2 Ancora su listener e visitor

Ora ci assegniamo due compiti ed useremo listener e visitor per fare due cose diverse:

- Numerare le righe
- Interpretare un linguaggio

Il linguaggio a cui faremo riferimento e' il linguaggio aritmetico con nomi di variabili ed assegnamenti. Una sequenza lineare di istruzioni in cui le istruzioni sono assegnamenti, stampe o espressioni.

Vogliamo contare gli assegnamenti, lo faremo con un listener, sappiamo che ANTLR ci da' un listener che e' un interfaccia da estendere, noi lo estendiamo e implementiamo i metodi che ci interessano. In questo caso ci interessa il numero di assegnamenti, quindi implementiamo il metodo che ci interessa. Poi creiamo un oggetto del listener e lo passiamo al walker di liblet.

Ora vogliamo interpretare il programma, un compito cosi' sofisticato spesso si realizza piu' facilmente con un visitor, perche' quando scrivo un linguaggio ho in testa per ogni nodo del linguaggio cosa voglio fare, in particolare noi vogliamo che per ogni nodo restituisca il valore del nodo, questo lo puo' fare facilmente se fa una visita in post-ordine conoscendo i valori dei figli.

In questo caso abbiamo solo variabili locali, quindi la cosa piu' facile da fare e' tenere una memoria che e' un dizionario di python, se vediamo un assegnamento prendiamo il nome della variabile, facciamo lo scarico ricorsivo chiamando expr e salviamo tutto in memoria. Se vediamo una variabile, facciamo un lookup in memoria e restituiamo il valore. Se vediamo un numero lo restituiamo direttamente, da notare che quando restituiamo l'intero e' l'unico passo interpretativo perche' stiamo trasformando testo in intero. Abbiamo un'espressione che fa la print, non ritorna niente ma fa una print. Dopo di che c'e' la parte di associazione, di nuovo sono divise perche' sono cose divise, in ogni caso visito tramite scarico ricorsivo il lato sinistro e destro e poi guardo il tipo dell'operatore, la visita delle parentesi e' un trucco sintattico che serve ad indicare diverse precedenze, quindi dal punto di vista interpretativo non serve a niente e si puo' solo mangiare.

Estendere questo parser su linguaggi piu' difficili e' immediato, la logica e' sempre la stessa, basta estendere le regole di parsing e le regole di interpretazione. Potrebbe essere un buon esempio. Il trucco cruciale e' che il visitatore e' ricorsivo quindi posso quando ho un dubbio su che cosa fare posso usare la ricorsione, per valutare un'espressione valuto ricorsivamente i figli e poi faccio l'operazione. Il problema di questa cosa e' che la pila di chiamate ricorsive puo' esplodere e non e' facile da tenere sotto controllo, soprattutto se vogliamo dare degli errori durante il parsing. Quindi abbiamo bisogno di fare una traduzione in cui rendo la ricorsione esplicita, significa liberarsi della ricorsione passando ad una struttura iterativa, significa avere una pila di chiamate esplicita invece della pila di record di attivazione, l'idea qui e' che tengo dei risultati parziali. Qui dobbiamo scegliere tra visitor che usa la ricorsione e listener in cui teniamo una pila esplicita per le chiamate.

L'idea in questo caso e' di tenere gli operandi sulla pila e quando siamo pronti a fare l'operazione togliamo gli operandi e mettiamo il risultato intermedio. Quindi il listener avra' due strutture dati per funzionare, una memoria per le variabili ed una pila. Qui ho

bisogno degli exit perche' voglio sapere cosa fare con quello che c'e' sulla pila dopo che ho guardato i figli. Se usciamo da un assegnamento significa che ho gia' valutato l'espressione e sulla pila c'e' il valore da assegnare, quindi assegniamo in memoria. Se usciamo da un id o un intero significa che valutiamo il valore e mettiamo sulla pila il valore. Quando usciamo da una divisione mi aspetto che i valori dei suoi operandi siano sulla pila messi dalla visita in post-ordine, quindi li tolgo dalla pila e metto il risultato della divisione sulla pila. Se usciamo da una somma invece mettiamo il risultato della somma sulla pila.

4.3 Dall'albero di parsing all'AST

L'idea e' di partire da un albero di parsing che ha tanti artefatti, toglierli e raggiungere una forma compatta che e' un abstract syntax tree. Per farlo usiamo una funzione ricorsiva.

Abbiamo visto che in ANTLR si puo' mettere del codice direttamente nella grammatica, non lo faremo mai e' troppo doloroso, poi non ha troppo senso tenere legata la grammatica con l'implementazione del parser.

4.3.1 Dispatch table

Useremo una dispatch table per capire cosa fare quando troviamo una regola (nella table ci saranno le funzioni ricorsive). Inizialmente diciamo che quando troviamo una regola e c'e' nella dispatch table chiamo la sua regola, se non la trovo metto tutti in un catchall:

Devo riempire la tabella, in questo caso definiamo le funzioni che ci servono quando vediamo gli operatori, da notare che come scarico ricorsivo non usano se stesse ma chiamano la funzione che usa la dispatch table, poi popolo la table:

Si e' visto che liblet implementa una classe TreeWalker che fa questo lavoro della funzione ricorsiva, possiamo dirgli dato un albero che attributo considerare, registrare delle funzioni che usa quando incontra dei simboli (con un annotazione o registrarle esplicitamente), inoltre permette anche di avere un catchall (ne esistono di tanti tipi).

Vediamo un esempio in cui eseguiamo il programma partendo da un ATS, notiamo che occupandoci degli atomi abbiamo solo const e var, non abbiamo bisogno di castare ad Int perche' lo abbiamo gia' fatto nella costruzione dell ATS quando mettevamo dentro il valore di una costante.

4.4 Trasformare

Vediamo alcuni esperimenti di trasformazione, in particolare inizialmente vogliamo trasformare un documento JSON in un documento HTML usando ANTLR.

Quello che facciamo noi e' modularizzare il processo, partiamo generando l'AST, si fa con un tree walker ricorsivo che ha la tabella di dispatch che discrimina i nodi in base al nome.

4.4.1 Trasformare HTML in matrici

L'idea e' che ho acceduto ad una pagina HTML con dentro delle tabelle e io voglio farmi una matrice che rappresenta i dati, e' una trasformazione di dominio.

Per farlo partiamo con il definirci una nostra grammatica invece di usare quella completa assumendo per semplicita' che dentro i td possano esserci solo interi.

Il processo e' sempre lo stesso, arriviamo all'AST e qui la cosa che cambia e' che per fare la trasformazione usiamo una struttura dati esterna, uno stack, quando vediamo i dati li pushiamo dentro lo stack e quando vediamo una row togliamo i figli e li mettiamo dentro un array, alla fine dello stack avremo un array di array che rappresenta la matrice.

Abbiamo visto anche che per costruire la tabella potremmo pensare di decorare i nodi con il numero di riga e colonna mettendo nella radice il numero di righe e colonne, cosi' posso poi precomputare una matrice di dimensioni fisse piena di None e poi riempirla visitando l'AST.

4.5 Traspilazione

L'idea e' quella di prendere un linguaggio e trasformarlo in un altro linguaggio. Questa cosa ha origini storiche perche' ad esempio il C non veniva compilato ma traspilato in Fortran. Ci sono anche logiche di portabilita' e di ottimizzazione. La portabilita' e' moderna, uno dei linguaggi dominanti nel web e' JavaScript, il problema di questo linguaggio e' che non e' standardizzato e non coinvolge tutti i browser. Ragione per cui e' nata una famiglia di traspilatori che prendono un JS recente e lo trasformano in un JS piu' vecchio che gira su tutti i browser. Un esempio e' Babel, che e' un traspilatore di JS, ma ce ne sono anche altri. Se esistono costrutti che nelle vecchie versioni non esistono si usano gli SHIM. E' nato anche typescript, che non esiste nei browser, ma viene traspilato in JS.

Noi partiamo da un linguaggio semplice SimpleLang, prima lo traspiliamo in JavaScript (facendo un text to text traspilazione). L'altro passo che faremo e' quello di sfruttare il fatto che alcuni linguaggi hanno una rappresentazione di alto livello, ma che e' un po' piu' snella di un testo (non richiede ulteriori processi di parsing) che puo' essere compilata dalla VM e generata senza troppo sforzo, quindi tradurremo questo linguaggio verso Python facendo una traduzione da testo verso l'AST di python (AST = Abstract Syntax Tree).

Dobbiamo decidere come fare l'IO, perche' ha senso compilare un programma solo se c'e' un input. Noi per farla facile cerchiamo di provvedere modi banali per dare l'input ai nostri programmi, l'interprete preleva delle cose dal mondo esterno e le inietta nel programma. Nel nostro caso assumiamo che ci siano tre variabili INPUT e per farla facile faremo che ha un solo OUTPUT. (dice che il progetto di quest'anno non avra' bisogno di librerie esterne).

Arrivati infondo alla traspilazione dobbiamo cercare di valorizzare la variabile INPUT, facciamo una funzione che cerca tutti gli input e per ciascuno di essi apre un prompt che prende l'input.

Vediamo ora come cercare di evitare il passo di parsing del linguaggio in cui arriviamo. Scriviamo il linguaggio in una forma in cui l'interprete di python sia in grado di interpretarlo direttamente, python perche' e' fornisce delle API per interagire con gli AST. Si puo' usare eval per fare valutare una funzione e generare un parse tree. una volta che

abbiamo un AST possiamo compilarlo con la funzione `compile` di python che vuole sapere cosa sta compilando e in che modalita'. Per fare la traspilazione verso l'ATS di python dobbiamo visitare il nostro albero e trasformare i nostri nodi nei loro corrispondenti nodi di python.

Dopo siamo di nuovo nella situazione di dover valorizzare gli input, i programmi di python hanno un dizionario dove pesca le variabili locali, noi possiamo aggiungere con un dizionario di python le variabili che ci servono. Ora li stiamo mettendo a mano noi, potremmo anche pensare di mettere delle funzioni per valorizzare le variabili di input.

4.6 Symbol table

Assomiglia ad un dizionario, abbiamo due funzioni `Bind` e `Look up` con cui mettiamo dentro un simbolo e lo cerchiamo. Hanno anche la competenza di entrare ed uscire da un contesto (`ENTER`, `EXIT`) come contesto potremmo intendere classi o funzioni.

4.6.1 Scope

Nei linguaggi di programmazione moderni le variabili non solo visibili ovunque, ma hanno uno scope. Per farlo utilizzeremo delle opportune symbol table. Esiste uno scoping locale che serve a correggere uno scoping globale. Possiamo avere anche uno scope unico o distinto posso usare lo stesso simbolo in contesti diversi o meno (una variable che si chiama come una funzione).

4.6.2 Implementazione symbol table

Esistono svariate implementazioni, noi ne vedremo due.

Implementazione deterministica: e' la piu' facile ed adoperiamo una pila di coppie (simbolo, informazione), fare il bind di un simbolo significa fare il push della coppia simbolo informazione. Il lookup e' la ricerca in questa pila dall'alto verso il basso. Enter spingo nella pila qualcosa che sono sicuro di riconoscere tipo `null`, `null`, in modo tale che quando faccio exit posso fare una catena di pop fino a che non trovo il separatore, sostanzialmente metto un separatore per i contesti. E' un po' inefficiente perche' quando faccio un lookup devo scorrere tutta la pila, e anche l'exit non e' bellissimo.

Possiamo vedere anche la symbol table come albero di dizionario, avremo dei dizionari (chiave - valore) attaccati al loro papa'. Quindi in questo caso dobbiamo fare una ricerca al massimo lineare nel numero di scope, poi dentro i dizionari la ricerca e' $O(1)$. La cosa bella e' che quando faccio un enter creo un dizionario vuoto e lo metto come figlio del dizionario padre, quando faccio un exit lo tolgo. In questo caso non abbiamo bisogno di mettere i separatori, perche' il dizionario padre e' il separatore.

4.7 Interpretazione

Siamo arrivati al punto in cui possiamo cominciare a parlare di interpretazione, siccome l'albero della AST e' una struttura ricorsiva, possiamo fare una visita ricorsiva dell'albero e interpretare il linguaggio. Questa e' la cosa piu' semplice da fare.

Dopo di che aggiungeremo le funzioni e penseremo a come interpretare le funzioni, con l'idea che le funzioni siano rientranti, quando esco dalla funzione ritorno da dove l'ho chiamata, inoltre i parametri della funzione sono trattati come variabili locali nello scope della funzione, per fare questo useremo la pila dei record di attivazione (chiamato anche call stack).

Come esempio abbiamo sempre il SimpleLang in cui abbiamo introdotto anche il prod-Expr. Vediamo come fare l'interprete ricorsivo, inchiodiamo una variabile globale e lo attacchiamo all'interprete, in python si possono incollare attributi agli oggetti, in questo caso lo facciamo con `interpreter.GLOBAL_MEMORY`. Il difetto dell'interpretazione ricorsiva e' il costo di esecuzione. Non abbiamo fatto robe molto innovative.

4.7.1 Funzioni

Vogliamo dare una nuova espressivita' al linguaggio, vogliamo dare un nome ad un pezzo di codice per eseguirlo piu' volte. Una funzione ha una collezione di parametri (formali) che operano dal punto di vista concettuale come variabili locali di quella funzione. Dobbiamo definire due momenti:

- Definizione di funzione: nome, parametri formali, corpo
- Invocazione di funzione: nome, parametri concreti (actual) che sono di fatto delle espressioni, potremmo avere il valore di ritorno potremmo quindi dividere le funzioni tra quelle che calcolano un valore a quelle che non lo fanno.

In quasi tutti i linguaggi un'espressione costituisce uno statement e noi faremo cosi', inoltre ci occuperemo solamente di funzioni di primo livello, non ci occuperemo di chiusure.

Ci interessa ora vedere come usare la pila di chiamate. Inanzitutto abbiamo deciso che un programma e' una sequenza di funzioni, decidiamo che ci sia una funzione che eseguiamo prima delle altre (main), dobbiamo inserire nella grammatica la definizione di funzione con i parametri formali e nell'invocazione delle espressioni possono esserci altre espressioni. Le espressioni diventano statement e ci mettiamo il return statement che e' una comoda convenzione per indicare il valore di ritorno della funzione, questo return e' opzionale se non c'e' return assumeremo che la funzione ritorni None.

Ora vediamo cosa fare quando visitiamo il nodo di dichiarazione di funzione, puo' succedere che ci siano i parametri formali o no, dobbiamo salvarci il nome della funzione e i parametri formali. Da stare attenti che nel caso in cui ci siano i parametri devo andare di due in due perche' devo saltare la virgola.

La chiamata e' sostanzialmente diversa, di nuovo posso avere gli argomenti o meno (anche in questo caso devo saltare la virgola). In questo caso devo creare un nuovo albero che chiama la funzione.

Adesso e' il momento di occuparci della chiamata. Dobbiamo predisporre un record di attivazione, che e' un dizionario in cui mettiamo i parametri formali e i valori dei parametri concreti. Poi dobbiamo fare l'enter della funzione, quindi dobbiamo mettere il record di attivazione nella pila dei record di attivazione, poi dobbiamo visitare il corpo della funzione. Alla fine della visita della funzione dobbiamo fare l'exit della funzione, quindi togliere il record di attivazione dalla pila e restituire il valore di ritorno. E' una pila perche' man mano che chiamo aggiungo record e quando ritorno faccio il pop del record di attivazione. Quindi valutiamo le espressioni dei parametri concreti, prepariamo

il record di attivazione, facciamo il push del record di attivazione nella pila, passiamo il controllo, quando ho finito di fare faccio il pop del record di attivazione e restituisco il valore di ritorno.

Aggiungiamo un altro attributo globale `ACTIVATION_RECORDS` che è una pila. La modifica cruciale che invece di cercare i valori nella memoria globale li cerchiamo nella pila di attivazione.

L'interpretazione del programma diventa la raccolta dell'interpretazione delle funzioni, le raccogliamo in un dizionario `FUNCTIONS` che indicizziamo con il nome delle funzioni e dentro ci mettiamo gli alberi di parsing delle funzioni. Interpretare il programma significa fare il visit del main, che come detto prima metterà in pila il record di attivazione, a sua volta metterà in pila i record di tutte le funzioni dopo main.

Il caso dell'istruzione `return` è gestito aggiungendo una variabile locale dal nome `_retval` per conservare il valore restituito.

Nella chiamata per preparare il record di attivazione dobbiamo valutare i parametri concreti con le espressioni (che sono proprio numeri nel nostro caso), il record è un dizionario di coppie.

Abbiamo anche visto come fare il ritorno forzato quando abbiamo un `return`, ad esempio in un `for` o in un `if`, quello che va fatto semplicemente è che quando valutiamo quelle espressioni non andiamo giù dritto come al solito, ma dopo il `visit(stat)` controllo che non ci sia `_retval` nella memoria, se c'è faccio `return` anche dal `for` o dall'`if`.

4.8 Tipi

Esistono errori `untrapped` o `trapped`. L'obiettivo è avere un codice `safe` e `well-behaved`, per fare questo ci vengono in aiuto i tipi e tutte le tecniche che ci permettono di capire se il programma è `safe`.

Alcuni linguaggi non hanno tipi ed altri sono tipizzati, in vario modo misura, nel senso che ogni istanza di variabile o funzione è decorata con un tipo, questo si chiama tipi espliciti. Ci sono altri sistemi in cui i tipi non sono espliciti, ma sono impliciti, nel senso che il compilatore o l'interprete si occupa di dedurre il tipo della variabile o della funzione. Quello che vogliamo vedere adesso è se un programma è `well-typed`, è ragionevole pensare che se un programma è ben tipato allora è anche `safe`, in realtà questa implicazione è delicata perché oltre ad avere un sistema di tipi dobbiamo averlo `sound` (ben fatto). Attenzione che la `soundness` nei linguaggi comuni non è garantita.

Se viviamo nel mondo dei linguaggi tipati espliciti possiamo usare il `typechecker` che verifica la `well-behavior` del programma in fase statica. Invece se siamo nel mondo dei linguaggi `untyped` o `typed implicit` dobbiamo fare un controllo dinamico. Tipicamente si usano tecniche miste.

4.8.1 Typechecking

Quello che faremo è decorare l'AST con delle `symbol table`, in modo tale che quando visitiamo l'AST possiamo vedere se il programma è ben tipato o meno. La cosa bella è che la `symbol table` è una struttura dati che ci permette di tenere sotto controllo i tipi delle variabili e delle funzioni, quindi possiamo usarla per fare il `type checking`.

Siamo in un mondo con solo tipi primitivi, solo interi e stringhe. Possiamo fare tutte le espressioni binarie con entrambi gli operandi interi, confrontare due stringhe oppure sommare due stringhe (ottenendone la concatenazione) o moltiplicare una stringa per un intero (ottenendone la ripetizione). L'enforcing di queste regole non può appartenere all'analisi lessicale del linguaggio, ma deve appartenere ad un'analisi successiva.

4.9 Interprete Iterativo

Vorremmo ora vedere un modo per linearizzare l'albero di parsing, in modo tale da non dover usare la ricorsione per interpretare il linguaggio. L'idea è quella di usare una pila per tenere traccia delle chiamate e dei risultati intermedi. In questo modo possiamo evitare di usare la ricorsione e rendere l'interprete più efficiente.

4.9.1 Code threading

L'operazione che impariamo a fare è il code threading (nel senso di filo sartoriale), l'idea è di tirare un filo attraverso i nodi dell'albero di parsing, poi se taglio gli archi e tengo i nodi che ho infilato nel filo ho una struttura linearizzata, è una rappresentazione topologica dell'albero in modo tale che posso camminare su questa struttura iterativa eliminando la ricorsione. Decoriamo i nodi con un campo che punta al nodo successivo (anche più campi se ci sono più figli).

È un ulteriore passo dopo l'AST decorato, la decorazione del nodo la facciamo in modo facile, lo decoriamo con un attributo `_thread_` e ci mettiamo dentro un nodo, da dire è facile, il problema di questa cosa è che la funzione di hash è mezza complicata, la cosa che dovremmo fare noi è magari non usare gli alberi di liblet ma usare una roba ad oggetti ed il puntatore al nodo usare un numero, non l'hash del nodo, in modo da evitare la ricorsione infinita.

La cosa cruciale è concatenare due fili, è il concatenamento di due liste. È comodo avere due nodi speciali BEGIN e END per indicare all'interprete dove cominciare e dove finire. Il modo più facile di fare il concatenamento è una visita in post-ordine, in cui concatenano i nodi figli e poi concatenano il nodo padre. Vediamo che così facendo torniamo anche da program e block, ma sono solo dei segnaposti, non ci interessano, possiamo evitarli.

Selezione

In un if-else vogliamo aggiungere un nodo di unione, valutiamo la condizione, torniamo al nodo if-else così l'interprete valuta se la condizione se è true va al true altrimenti al false, gli alberi dei true e false vanno ad un nodo JOIN.

Iterazione

Funziona in modo abbastanza analogo, il filo nella condizione valuta se deve eseguire ancora il ciclo o meno.

4.9.2 Interprete

Per prima cosa, semplifichiamo i fili tenendo da parte solo le istruzioi ed i salti, iniziamo col trasformare ogni nodo (da usare nei salti) in un numero progerssivo, dopo aver numerato i nodi.

Ho bisogno di strutture dati, memoria globale, stack e numero di istruzione (IP, instruction pointer). Uso il trucco di tenere in cima alla pila quando c'e' un repeat il numero di volte che devo ripetere il ciclo.

Chapter 5

LLVM

L'idea di llvm è quella di avere un linguaggio di rappresentazione intermedia (IR) che sia indipendente dalla piattaforma, in modo da poter scrivere compilatori che possano generare codice per diverse architetture senza dover riscrivere tutto il codice del compilatore.

La strategia che è stata scelta non è generare un AST generale che possa andare bene per qualsiasi architettura, ma piuttosto generare un IR (Intermediate Representation) che è una cosa simile all'assembler ma con un livello di astrazione superiore, in modo da poter essere facilmente trasformato in codice macchina per diverse architetture.

Quello che vediamo oggi è come trasformare un AST in IR, ma non ci occuperemo di come trasformare l'IR in codice macchina, perché questo è un argomento che richiede una conoscenza approfondita dell'architettura della macchina su cui si vuole generare il codice.

Una cosa cruciale è che nell'IR c'è un numero infinito di registri, è cruciale perché laddove avevamo usato una pila per gestire le espressioni e le variabili, questa cosa ci permette di non dover gestire la pila, ma di poter usare i registri direttamente e questo permette di tradurre le espressioni molto più facilmente. Questo rende l'IR molto più semplice da gestire e da trasformare in codice macchina. Noi non entreremo nel merito però la cosa tosta è capire come passare da una rappresentazione con infiniti registri a una rappresentazione con un numero finito di registri, che è quello che si fa nella fase di register allocation.

5.1 IR di LLVM

Cerchiamo di capire come funziona l'IR di LLVM. Vediamo un sottoinsieme delle istruzioni che sono disponibili in LLVM IR, che sono le istruzioni che ci interessano per il nostro progetto. Come obiettivi di implementazione l'IR aveva di essere semplice da generare e da leggere, e di essere facilmente trasformabile in codice macchina per diverse architetture. Deve in qualche modo rappresentare l'HW. Non deve perdere completamente di informazione, affinché la traduzione verso il codice macchina sia possibile non deve opacizzare completamente le informazioni, deve consentire le ottimizzazioni.

Un pezzo di codice LLVM è fatto da una serie di informazioni, una delle quali è verso dove stiamo traducendo, poi ci sono alcune parti che sono tipiche di un linguaggio di programmazione, come le dichiarazioni delle variabili globali etc, dichiarazioni delle funzioni e poi il body delle funzioni. Noi non adopereremo le variabili globali, avremo solo interi e avremo solo una funzione main (nel progetto servono altre funzioni però).

Le funzioni sono fatte da un'intestazione che ci dice il nome, cosa restituiscono e gli argomenti, la cosa cruciale che hanno i blocchi nominati (che si possono scegliere liberamente), noi dobbiamo dobbiamo fare in modo che i blocchi siano nominati perché ANTLR ci dà tutto il contesto.

I blocchi sono le istruzioni del linguaggio, hanno una quantità infinita di registri, significa che si può usare %n per indicare il registro dove mettere le informazioni. Ogni istruzione ha un tipo. La cosa cruciale è che i registri sono infiniti ma non si possono riassegnare. Questa cosa dei registri si chiama SSA (Static Single Assignment), significa che ogni variabile viene assegnata una sola volta, e questo rende più semplice l'analisi del codice e le ottimizzazioni.

Chapter 6

Progetto

L'idea e' quella di ragionare sulla ricorsione investigando la questione che la ricorsione e' una cosa potente ma costosa. Saltino e' un linguaggio che non ha iterazione, se non ho iterazione devo fare la ricorsione. Il punto e' che esistono una serie di strategie per mitigare questo approccio, un esempio di somma senza ricorsione:

```
1  Somma(n){
2      if n == 0 {
3          return 0
4      } else {
5          return n + Somma(n-1)
6      }
7  }
```

Quando la ricorsione e' l'ultimo passo e' facile trasformare la ricorsione in iterazione. Questa cosa si chiama tail recursion, elimini la ricorsione con un while true in cui riassegno quello che ho sulla pila dello stack nel caso della ricorsione, deve essere l'interprete a fare questa cosa.

Quindi l'idea e' inizialmente implementare un interprete ricorsivo e poi implementare un linguaggio saltone che non ha ricorsione, eliminando la ricorsione con un while true eliminando la ricorsione di coda. Ocio che con la somma non funziona quella cosa che dicevamo, la ricorsione non e' l'ultima cosa che faccio, devo ritornare i risultati parziali, ma devo ritornare degli accumulati, di fatto mi porto la somma parziale come secondo argomento:

```
1  Somma(n, acc){
2      if n == 0 {
3          return acc
4      } else {
5          return Somma(n-1, n+acc)
6      }
7  }
```

A questo punto posso scrivere con un while true:

```
1  Somma(n){
2      acc = 0
3      while true {
4          if n == 0 {
5              return acc
6          } else {
7              new_n = n - 1
8              new_acc = n + acc
9              n = new_n
10             acc = new_acc
11         }
12     }
13 }
```

Ha detto che fare l'interprete iterativo rende molto facile la trasformazione in saltone, ovvero togliere la ricorsione.

Gli errori devono essere trappati, non devono vedersi stack trace di python.